

算法竞赛核心模板库

Jiangcheneee　　华中农业大学

2026 年 10 月 29 日

目录

1 一键启动模板 **3**

1.1 启动文件 3

2 stl 库 **4**

2.1 algorithm 4
2.2 atan2 7
2.3 bitset 8
2.4 map 11
2.5 pbds_tree 13
2.6 priority_queue 15
2.7 queue 17
2.8 set&&multiset 19
2.9 stack 21
2.10 string 23
2.11 tuple 26
2.12 vector 27

3 博弈论 **30**

3.1 sg 函数 30

4 图论 **31**

4.1 dij 31
4.2 欧拉路径 32

5 数学 **34**

5.1 FFT 34
5.2 NTT 35
5.3 exgcd 37
5.4 三分 38
5.5 中国剩余定理 40
5.6 中国剩余定理（推导笔记） 42
5.7 任意模数 NTT 45
5.8 前缀基 47
5.9 前缀线性基 48
5.10 卢卡斯定理 51
5.11 卢卡斯定理（推导笔记） 54
5.12 多项式卷积（推导笔记） 56
5.13 欧拉筛 59

5.14 矩阵快速幂 . . . 60
5.15 矩阵快速幂（加速 DP 取 max） . . . 61
5.16 线性基（方案还原与删除） . . . 64
5.17 线性基（贪心法） . . . 67
5.18 范德蒙德卷积（推导笔记） . . . 70
5.19 莫比乌斯反演（二维 GCD 求和） . . . 73
5.20 莫比乌斯反演（推导笔记） . . . 74
5.21 莫比乌斯反演（特定 GCD 计数） . . . 78
5.22 行列式计算 . . . 80

6 数据结构 82
6.1 01trie . . . 82
6.2 dsu . . . 83
6.3 主席树（区间 MEX） . . . 85
6.4 主席树（区间元素数） . . . 86
6.5 主席树（区间第 K 小） . . . 87
6.6 区间 mex . . . 88
6.7 树状数组 . . . 91
6.8 线段树 . . . 92
6.9 线段树（区间 max） . . . 94
6.10 莫队 . . . 96

7 树论 98
7.1 倍增 . . . 98
7.2 启发式合并 . . . 99

8 计算几何 102
8.1 凸包（单调链法） . . . 102
8.2 极角排序 . . . 105
8.3 求凸包直径（旋转卡壳） . . . 107
8.4 求凸包面积 . . . 111
8.5 闵可夫斯基和与皮克定理 . . . 111

9 杂项 113
9.1 CDQ 分治 . . . 113
9.2 CDQ 分治优化 DP . . . 115
9.3 CDQ 分治（推导笔记） . . . 118
9.4 int128 . . . 122
9.5 modint . . . 123

1 一键启动模板

1.1 启动文件

```cpp
#pragma GCC optimize("O2")
#pragma GCC optimize("O3")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp> // 包含所有容器（哈希表、树、堆）
#include <ext/pb_ds/tree_policy.hpp>     // 包含树的相关策略（用于实现平衡树）
#include <ext/pb_ds/hash_policy.hpp>     // 包含哈希表的相关策略
//#include <ext/pb_ds/priority_queue.hpp>  // 包含优先级队列（堆）
using namespace std;
using namespace __gnu_pbds;
using ll = long long;
template <typename T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define sm int,vector<int>,greater<int>
#define int long long
mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
const int N=1e6+5,mod=998244353,inf=1e9+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
int gcd(int x,int y)
{
    if(y==0)return x;
    return gcd(y,x%y);
}
int ksm(int a,int n,int p)
{
    if(n<0)return 0;
    int res=1;
    while(n)
    {
        if(n&1)res=res*a%p;
        n>>=1;
        a=a*a%p;
    }
    return res;
```

```
}
void solve(){

}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--){
        solve();
    }
//  system("x-www-browser https://ys.mihoyo.com/");
//  cout << fixed << setprecision(10) << x << '\n';
    return 0;
}
/*
1
3411 4134 32 23


*/
```

2 stl 库

2.1 algorithm

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. 排序与极值
// ==========================================
void algo_sort_extrema() {
    vector<int> a = {5, 1, 4, 2, 3, 4};

    // 1. 稳定排序
    // 关键字相等的元素保持原有相对顺序。
    stable_sort(a.begin(), a.end());

```

```cpp
    // 2. 线性时间求第 K 小 (O(N))
    // 用于求中位数或前 K 小，不要求整体有序。
    // 执行后 a[2] 为第 3 小的数，左侧元素均不大于它，右侧元素均不小于它。
    nth_element(a.begin(), a.begin() + 2, a.end());

    // 3. 一次遍历同时求最小值与最大值
    auto [min_it, max_it] = minmax_element(a.begin(), a.end());
    cout << "Min: " << *min_it << ", Max: " << *max_it << endl;
}

// ==========================================
// 2. 全排列与序列变换
// ==========================================
void algo_transform() {
    vector<int> p = {1, 2, 3};

    // 1. 下一个全排列 (适用于 N 较小的枚举)
    // 使用前数组必须升序。
    do {
        // 依次产生: 123, 132, 213, 231, 312, 321
    } while (next_permutation(p.begin(), p.end()));

    // 2. 区间循环移位
    // 使 middle 成为新的首元素，[first, middle) 移到末尾，O(N)。
    vector<int> v = {1, 2, 3, 4, 5};
    // 以 v.begin() + 2 (即数字 3) 为新的首元素
    rotate(v.begin(), v.begin() + 2, v.end());
    // 结果 v = {3, 4, 5, 1, 2}
}

// ==========================================
// 3. 有序数组的集合操作
// ==========================================
// 求两个已升序数组的交集、并集、差集，复杂度 O(N + M);
// 用 std::set 实现需要 O(N log N)。
void algo_set_ops() {
    vector<int> v1 = {1, 2, 4, 5, 7};
    vector<int> v2 = {2, 3, 5, 6};
    vector<int> res(10); // 需提前分配足够大的空间

    // 求交集，返回结果区间的尾后迭代器
    auto it = set_intersection(v1.begin(), v1.end(), v2.begin(), v2.end(),
        res.begin());
```

```
    // 删除多余空间
    res.resize(it - res.begin());
    // 此时 res = {2, 5}

    // 其他集合操作:
    // set_union (并集: 1, 2, 3, 4, 5, 6, 7)
    // set_difference (差集 v1 - v2: 1, 4, 7)
}

// ==========================================
// 4. numeric 常用函数
// ==========================================
void numeric_magic() {
    vector<int> a(5);

    // 1. iota: 从初值开始递增填充
    // 常用于并查集初始化 fa[i] = i。
    iota(a.begin(), a.end(), 1);
    // 结果 a = {1, 2, 3, 4, 5}

    // 2. accumulate: 区间求和
    // 第三个参数的类型即累加使用的类型。
    vector<int> big_nums = {1000000000, 1000000000, 1000000000};

    // 按 int 累加, 会整型溢出
    long long sum_wrong = accumulate(big_nums.begin(), big_nums.end(), 0);

    // 写成 0LL, 按 long long 累加
    long long sum_right = accumulate(big_nums.begin(), big_nums.end(), 0LL)
        ;

    // 3. gcd / lcm (C++17 起可用)
    int g = std::gcd(12, 18); // 返回 6
    int l = std::lcm(12, 18); // 返回 36
}

// ==========================================
// 5. 填充与离散化
// ==========================================
void fill_and_discretize() {
    // 1. fill: 按值填充
    // memset 按字节填充, 只能填 0、-1、0x3f 等; 填 100 这类值须用 fill。
    vector<int> dp(100);
```

```
    fill(dp.begin(), dp.end(), 100);

    // 2. 离散化
    // 值域 1e9、元素个数 1e5 的线段树、树状数组题目常用。
    vector<int> vals = {500, 10, 1000000000, 10, 500, 999};

    // 第一步：排序
    sort(vals.begin(), vals.end());
    // 第二步：去重
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    // 第三步：查询 500 离散化后的编号，从 1 开始
    int old_val = 500;
    int new_id = lower_bound(vals.begin(), vals.end(), old_val) - vals.
        begin() + 1;
    // 结果 new_id = 2
}
```

2.2 atan2

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 计算几何：atan2 安全极角与扫描线极角排序
// ==========================================
const double PI = acos(-1.0);
const double eps = 1e-9;

// 浮点数比较函数（防精度误差）
int sgn(double x) {
    if (abs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;

    // 核心改造：将 atan2 的 [-PI, PI] 强制映射到 [0, 2*PI)
    // 这样在极角扫描时角度单调递增，不会出现负数
    double angle() const {
        double ang = atan2(y, x);
        if (sgn(ang) < 0) ang += 2 * PI;
```

```
        return ang;
    }

    // 到原点的距离平方
    double dist2() const {
        return x * x + y * y;
    }
};

// ========================================
// 浮点数极角排序比较器
// ========================================
bool cmp_atan2(const Point& a, const Point& b) {
    double ang1 = a.angle();
    double ang2 = b.angle();

    // 如果极角不同，按极角升序
    if (sgn(ang1 - ang2) != 0) {
        return ang1 < ang2;
    }
    // 极角相同（共线），按距离原点从近到远排序
    return a.dist2() < b.dist2();
}

void atan2_example() {
    vector<Point> pts = {{1, 1}, {-1, 1}, {-1, -1}, {1, -1}};

    // 输出角度：转换为通俗易懂的度数 (180.0 / PI)
    for (auto p : pts) {
        cout << "点 (" << p.x << "," << p.y << ") 的极角是: "
        << p.angle() * 180.0 / PI << " 度\n";
    }
    // 预期输出：45度，135度，225度，315度（递增，无负数）

    sort(pts.begin(), pts.end(), cmp_atan2);
}
```

2.3 bitset

```
#include <bits/stdc++.h>
using namespace std;

// ========================================
```

```cpp
// 1. 构造与查询
// ==========================================
void bitset_basics() {
    // 大小必须是编译期常量，不能用变量
    const int N = 1005;
    bitset<N> bs1;              // 默认全 0
    bitset<N> bs2(13);          // 用十进制数初始化 (1101)
    bitset<N> bs3("101001");    // 用 01 字符串初始化

    // 状态统计
    int ones = bs3.count();     // O(N/64) 统计 1 的个数
    bool has_one = bs3.any();   // 是否至少有一个 1
    bool all_zero = bs3.none();// 是否全为 0
    bool all_one = bs3.all();   // 是否全为 1 (C++11)
}

// ==========================================
// 2. 修改与位运算
// ==========================================
void bitset_modifiers() {
    bitset<100> bs;

    // 单点修改 (O(1))
    bs.set(5);      // 将第 5 位设为 1 (最右边是第 0 位)
    bs.set(5, 0);   // 将第 5 位设为 0
    bs.reset(5);    // 等同于设为 0
    bs.flip(5);     // 翻转第 5 位

    // 整体修改 (O(N/64))
    bs.set();       // 全体设为 1
    bs.reset();     // 全体清零
    bs.flip();      // 全体取反

    // 位运算复杂度均为 O(N/64)，可当作集合运算使用
    bitset<100> a("1100"), b("1010");
    auto c = a & b; // 交集
    auto d = a | b; // 并集
    auto e = a ^ b; // 异或 (集合对称差)
    auto f = a << 3;// 整体左移
}

// ==========================================
// 3. 遍历所有 1 的位置 (GCC 专属)
```

```cpp
// ============================================
// 遍历复杂度与 1 的个数成正比；集合稀疏时优于逐位检查。

void bitset_iteration() {
    bitset<1000> bs("100100001000"); // 稀疏集合

    // 固定写法：
    // _Find_first() 返回第一个 1 的索引
    // _Find_next(i) 返回 i 之后的第一个 1 的索引
    // 找不到时返回 bs.size()
    for (int i = bs._Find_first(); i < bs.size(); i = bs._Find_next(i)) {
        cout << "在位置 " << i << " 发现了一个 1! " << endl;
    }
}

// ============================================
// 4. 背包可行性：O(NV / 64)
// ============================================
// n 个物品，体积 w[i]，求容量 V 内可凑出的体积；只判可行性，不求方案数。
void dp_bitset_knapsack() {
    vector<int> w = {2, 3, 7};
    int max_V = 10000;

    // dp[i] 为 1 表示体积 i 可以被凑出来
    bitset<10005> dp;
    dp.set(0); // 体积 0 总是合法的

    for (int weight : w) {
        // 单次转移 O(V/64)：已凑出的体积加上 weight 后仍可凑出
        dp |= (dp << weight);
    }

    if (dp[10]) cout << "能凑出体积 10" << endl;
}

// ============================================
// 5. 传递闭包：O(V^3 / 64)
// ============================================
// 有向图可达性。Floyd 为 O(V^3)，V = 2000 时超时，用 bitset 降至 O(V^3 / 64)。
void graph_transitive_closure() {
    const int V = 2005;
    bitset<V> reach[V]; // reach[i][j] 表示 i 能否到达 j
```

```

    // 假设图中存在边 (u, v)，则初始化 reach[u].set(v)

    // Floyd 算法
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            // i 可达 k 时，k 能到达的点 i 也能到达
            if (reach[i][k]) {
                reach[i] |= reach[k]; // 单次合并 O(V/64)
            }
        }
    }
}
```

2.4 map

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. map 基础
// ==========================================
void map_basics() {
    map<string, int> mp;

    // 基础插入
    mp["apple"] = 5;
    mp.insert({"banana", 3});

    // [] 访问不存在的键会自动插入默认值，所以不能用 mp["x"] > 0 判断存在性；
    // 循环中反复访问不存在的键会使 map 不断增长，可能 MLE 或 TLE。
    // 判断存在性应当用 count 或 find：

    if (mp.count("orange")) { // 返回 0 或 1
        // 存在
    }
    if (mp.find("orange") != mp.end()) {
        // 存在
    }
}

// ==========================================
```

```cpp
// 2. 有序性与二分
// ==========================================
void map_advanced() {
    map<int, string> mp;
    mp[10] = "A";
    mp[20] = "B";
    mp[30] = "C";

    // map 按键有序，可直接二分
    // 第一个键 >= 25 的元素
    auto it = mp.lower_bound(25);
    if (it != mp.end()) {
        cout << it->first << " " << it->second << endl; // 输出 30 C
    }

    // 按键由大到小逆序遍历
    for (auto it = mp.rbegin(); it != mp.rend(); ++it) {
        cout << it->first << " " << it->second << endl;
    }

    // 按键降序：传入 greater<int>
    map<int, string, greater<int>> mp_desc;
}

// ==========================================
// 3. unordered_map 与 pair 哈希
// ==========================================
// 标准库未为 pair 提供哈希函数，unordered_map<pair<int, int>, int> 无法通过编译，
// 需自行提供哈希函数。

struct pair_hash {
    template <class T1, class T2>
    size_t operator () (const pair<T1, T2>& p) const {
        auto h1 = hash<T1>{}(p.first);
        auto h2 = hash<T2>{}(p.second);
        // 组合两个哈希值，降低冲突概率
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
    }
};

void unordered_map_usage() {
    // 基础用法
```

```
    unordered_map<int, int> ump;
    ump[1] = 100;

    // 使用自定义 pair_hash
    // 常用于二维坐标的点权与有向边的权值
    unordered_map<pair<int, int>, int, pair_hash> edge_weight;

    edge_weight[{1, 2}] = 50; // 边 (1, 2) 的权值为 50

// 原生 std::unordered_map<int, int> 的哈希函数可被构造数据卡到单次操作 O(N)。
// 解决方式：使用 PBDS 的 gp_hash_table，或为 unordered_map 指定自定义哈希：
    // unordered_map<long long, int, custom_hash> safe_ump;
}
```

2.5 pbds_tree

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
using ll = long long;

template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void test_ordered_set() {
    ordered_set<int> st; // 元素类型可换成 ll 等
    st.insert(10);
    st.insert(30);
    st.insert(30); // 自动去重

    int rank = st.order_of_key(30); // 严格小于 30 的个数，返回 1
    int val = *st.find_by_order(1); // 排名第 2 的数，下标从 0 开始，返回 30
}

template <typename T>
struct ordered_multiset {
```

```
    ordered_set<pair<T, int>> tr;
    int timer = 0; // 区分相同元素的时间戳

    // 插入 x
    void insert(T x) {
        tr.insert({x, ++timer});
    }

    // 删除一个值为 x 的元素
    void erase(T x) {
        // timer 从 1 开始，{x, 0} 可定位到值恰为 x 的第一个元素
        auto it = tr.lower_bound({x, 0});
        if (it != tr.end() && it->first == x) {
            tr.erase(it);
        }
    }

    // 严格小于 x 的元素个数
    int rank(T x) {
        return tr.order_of_key({x, 0});
    }

    // 第 k 小的数，k 从 0 开始
    T k_th(int k) {
        return tr.find_by_order(k)->first;
    }

    // 值 <= x 的元素个数
    int count_less_equal(T x) {
        // 取 timer 的极大值，覆盖所有值为 x 的元素
        return tr.order_of_key({x, 2e9});
    }

    int size() { return tr.size(); }
    void clear() { tr.clear(); timer = 0; }
};

void test_ordered_multiset() {
    ordered_multiset<ll> ms; // 元素类型为 ll
    ms.insert(30ll);
    ms.insert(30ll);
    ms.insert(10ll);

```

```
    int r = ms.rank(30ll);        // 只有 10 严格小于 30，返回 1
    ll val = ms.k_th(1);          // 排名第 2 的元素，返回 30
    int cnt = ms.count_less_equal(30ll); // 三个数 <= 30，返回 3
}
```

2.6 priority_queue

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. 大根堆与小根堆
// ==========================================
void pq_basics() {
    // 1. 默认是大根堆，堆顶元素最大
    priority_queue<int> max_heap;
    max_heap.push(10);
    max_heap.push(30);
    max_heap.push(20);
    int top_max = max_heap.top(); // 30
    max_heap.pop();               // 弹出 30

    // 2. 小根堆需额外指定比较器，堆顶元素最小
    priority_queue<int, vector<int>, greater<int>> min_heap;
    min_heap.push(10);
    min_heap.push(30);
    min_heap.push(20);
    int top_min = min_heap.top(); // 10
}

// ==========================================
// 2. 存 pair
// ==========================================
void pq_pair() {
    // pair 默认先比较 first，再比较 second。
    // Dijkstra 中常用 first 存距离 dist、second 存节点编号 u。
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int
        , int>>> pq;

    pq.push({0, 1}); // 源点 1，距离 0
    pq.push({5, 2}); // 节点 2，距离 5

    auto [dist, u] = pq.top(); // C++17 结构化绑定
```

```
    pq.pop();
}

// ==========================================
// 3. 存结构体与比较规则
// ==========================================
struct Node {
    int u, dist;

    // 优先队列中若 a < b 为真，则 b 排在 a 前面，即更靠近堆顶。
    // 因此要让 dist 小的靠近堆顶，operator< 中需要写 dist > other.dist。
    bool operator<(const Node& other) const {
        // dist 小的靠近堆顶，故此处用大于号
        return dist > other.dist;
    }
};

void pq_struct() {
    // 结构体内已重载 <，此处按默认方式声明即可，
    // 其行为等价于以 dist 为关键字的小根堆。
    priority_queue<Node> q;

    q.push({1, 10});
    q.push({2, 5});
    q.push({3, 20});

    // 堆顶为 dist 最小的元素，即 {2, 5}
    cout << q.top().u << " " << q.top().dist << endl;
}

// ==========================================
// 4. 线性建堆
// ==========================================
void pq_linear_build() {
    vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

    // 做法一：逐个 push 入堆，复杂度 O(N log N)。
    priority_queue<int> q1;
    for(int x : v) q1.push(x);

    // 做法二：用迭代器区间构造，底层调用 std::make_heap，复杂度 O(N)。
    priority_queue<int> q2(v.begin(), v.end());
}
```

2.7 queue

```cpp
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. queue 基础用法
// ==========================================
void queue_basics() {
    queue<int> q;
    q.push(1); // 入队
    q.push(2);

    int head = q.front(); // 队头元素 (1)
    q.pop();              // 弹出队头，无返回值

    // std::queue 没有 clear()，多组数据下直接调用会编译报错。

    // 方式一：赋值为空队列
    q = queue<int>();

    // 方式二：与空队列 swap
    queue<int>().swap(q);

    // 方式三：while(!q.empty()) q.pop(); 常数较大
}

// ==========================================
// 2. deque 与 0-1 BFS
// ==========================================
// deque 支持在头部和尾部以 O(1) 插入和删除，并支持随机访问。
void deque_01_bfs() {
    deque<int> dq;

    // 基础操作：头尾均可进出
    dq.push_back(1);
    dq.push_front(2);
    dq.pop_back();
    dq.pop_front();

    // 边权只有 0 和 1 时可用 0-1 BFS 求最短路，
    // 复杂度 O(V + E)，优于 Dijkstra 的 O(E log V)。

```

```
    // 伪代码：核心逻辑
    /*
    dq.push_front(起点);
    while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto edge : graph[u]) {
    int v = edge.to, w = edge.weight; // w 只能是 0 或 1
    if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
    // 边权为 0 入队头，边权为 1 入队尾
    if (w == 0) dq.push_front(v);
    else dq.push_back(v);
    }
    }
    }
    */
}

// ==========================================
// 3. 单调队列
// ==========================================
// 用于求滑动窗口内的最大值 / 最小值，复杂度 O(N)，每个元素至多进出队列一次。
// 队列中存下标；进队前弹出队尾所有不大于 a[i] 的元素，队头即当前窗口的最值。

void monotonic_queue_example() {
    vector<int> a = {1, 3, -1, -3, 5, 3, 6, 7}; // 原数组
    int k = 3; // 滑动窗口大小为 3
    int n = a.size();

    deque<int> q; // 存储数组下标
    vector<int> max_res; // 存储每个窗口的最大值

    for (int i = 0; i < n; i++) {
        // 步骤 1：弹出已滑出窗口 [i-k+1, i] 的队头
        // q.front() <= i - k 时该下标已过期
        if (!q.empty() && q.front() <= i - k) {
            q.pop_front();
        }

        // 步骤 2：维护单调递减性。a[i] 入队更晚且不小于队尾元素，
        // 故队尾元素不可能再成为窗口最大值，可直接弹出。
```

```
        while (!q.empty() && a[q.back()] <= a[i]) {
            q.pop_back();
        }

        // 步骤 3: 新元素下标入队
        q.push_back(i);

        // 步骤 4: 窗口形成后记录答案，队头即最大值的下标
        if (i >= k - 1) {
            max_res.push_back(a[q.front()]);
        }
    }

    // 输出 max_res 即每个窗口的最大值
}
```

2.8 set&&multiset

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. set 基础与查找
// ==========================================
void set_basics() {
    set<int> s;
    s.insert(3);
    s.insert(1);
    s.insert(4);
    s.insert(1); // set 自动去重，容器中只有 1, 3, 4

    // std::lower_bound(s.begin(), s.end(), x) 对 set 是 O(N)，
    // 应使用 set 的成员函数，复杂度 O(log N)。
    auto it1 = s.lower_bound(3); // 第一个 >= 3 的迭代器
    auto it2 = s.upper_bound(3); // 第一个 > 3 的迭代器

    // 找前驱和后继
    if (it1 != s.begin()) {
        auto pre = prev(it1); // 前驱（小于它的最大元素）
    }
    if (it1 != s.end()) {
        auto nxt = next(it1); // 后继，也可写 ++it1
    }
```

```cpp

    // 遍历
    for (auto x : s) cout << x << " ";
}

// ==========================================
// 2. 自定义比较规则
// ==========================================
// 用于使 set 降序排列，或存储结构体。
struct Node {
    int l, r, val;
    // 按左端点升序，常用于 ODT
    bool operator<(const Node& other) const {
        return l < other.l;
    }
};

void set_custom() {
    // 内置类型改为降序排列
    set<int, greater<int>> s_desc;

    // 结构体 set
    set<Node> odt;
    odt.insert({1, 5, 10});
}

// ==========================================
// 3. multiset 与删除陷阱
// ==========================================
void multiset_traps() {
    // multiset 允许重复元素，常用于动态维护最值或中位数
    multiset<int> ms = {1, 2, 2, 2, 3};

    // 陷阱 1: erase(x) 会删除所有等于 x 的元素
    ms.erase(2);
    // 此时 ms 为 {1, 3}

    // 只删除一个元素应按迭代器删除
    multiset<int> ms2 = {1, 2, 2, 2, 3};
    auto it = ms2.find(2);
    if (it != ms2.end()) {
        ms2.erase(it); // 只删除该迭代器指向的一个元素
    }
```

```
    // 此时 ms2 为 {1, 2, 2, 3}

    // 陷阱 2: count() 的复杂度为 O(log N + K), K 为 x 的出现次数。
    // 元素大量重复时每次调用都可能超时。
    // 只判断存在性时改用 ms.find(x) != ms.end()。
}

// ==========================================
// 4. 用 set 维护区间（split 操作）
// ==========================================
// 利用 set 维护多个互不相交的区间。
set<Node> s; // Node 定义见上方
auto split(int pos) {
    // 找到第一个左端点 >= pos 的区间
    auto it = s.lower_bound({pos, 0, 0});
    // 已是某区间左端点则直接返回
    if (it != s.end() && it->l == pos) return it;

    // 否则 pos 一定在它前一个区间内部
    --it;
    int l = it->l, r = it->r, v = it->val;
    s.erase(it); // 删除原区间
    s.insert({l, pos - 1, v}); // 插入左半部分
    return s.insert({pos, r, v}).first; // 插入右半部分并返回其迭代器
}
```

2.9 stack

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. stack 基础用法
// ==========================================
void stack_basics() {
    stack<int> st;
    st.push(1); // 入栈
    st.push(2);

    int top_val = st.top(); // 获取栈顶元素 (2)
    st.pop();               // 弹出栈顶 (无返回值)

    bool is_empty = st.empty(); // 判断是否为空
```

```cpp

    // std::stack 不支持访问栈底元素，也不支持遍历。
}

// ==========================================
// 2. 数组模拟栈
// ==========================================
void array_stack() {
    const int N = 100005;
    int stk[N], top_idx = 0; // top_idx 表示栈中元素个数

    // 入栈 (push)
    stk[++top_idx] = 10;
    stk[++top_idx] = 20;

    // 访问栈顶 (top)
    int val = stk[top_idx];

    // 出栈 (pop)
    top_idx--;

    // 判断为空 (empty)
    bool empty = (top_idx == 0);

    // 数组模拟栈可直接从栈底到栈顶遍历
    for (int i = 1; i <= top_idx; i++) {
        cout << stk[i] << " ";
    }
}

// 也可用 vector 模拟栈：push_back() 入栈，pop_back() 出栈，back() 取栈顶。

// ==========================================
// 3. 单调栈
// ==========================================
// 求每个元素左侧 / 右侧第一个比它大或小的元素，复杂度 O(N)，每个元素至多进出栈一次。
// 经典题目：最大矩形面积（LeetCode 84）、视野总和。

void monotonic_stack_example() {
    vector<int> a = {2, 1, 5, 6, 2, 3};
    int n = a.size();

```

```
    // ans[i] 为 i 右侧第一个比 a[i] 小的元素下标，
    // 不存在这样的元素时保持初值 n。
    vector<int> ans(n, n);

    // 栈中存下标而非值
    stack<int> st;

    for (int i = 0; i < n; i++) {
        // 维护单调递增栈：a[i] 小于栈顶元素时，
        // 栈顶元素右侧第一个比它小的元素即为 a[i]。
        while (!st.empty() && a[st.top()] > a[i]) {
            int top_idx = st.top();
            ans[top_idx] = i; // 记录答案
            st.pop();         // 弹出栈顶
        }
        // 新元素下标入栈
        st.push(i);
    }

    // 遍历结束后栈中剩余元素右侧没有更小的数，ans 保持默认值 n。
}
```

2.10 string

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. 基础 I/O
// ==========================================
void string_io_traps() {
    string s;
    // 基础读取，遇空格或换行停止
    // cin >> s;

    // cin >> 与 getline 混用
    int n;
    // cin >> n;
    // 读入数字后再用 getline 读下一行：
    // getline(cin, s);
    // 结果 s 为空串：cin 不读走数字后面的换行符 '\n'。

    // 使用 getline 前先取走缓冲区中残留的换行符：
```

```
    /*
    cin >> n;
    string dummy;
    getline(cin, dummy); // 取走上一行的换行符
    // 或者用: cin.ignore(numeric_limits<streamsize>::max(), '\n');
    getline(cin, s);     // 读取含空格的一整行
    */
}

// ==========================================
// 2. 增删改与截取
// ==========================================
void string_modifiers() {
    string s = "Hello";

    // 尾部拼接, 复杂度 O(L)
    s += " World";
    s.push_back('!');  // 尾部追加单字符
    s.pop_back();      // 尾部删除单字符 (C++11)

    // substr 的第二个参数是长度, 不是结束下标。
    // 从下标 6 开始取长度 5 的子串, 结果为 "World"
    string sub1 = s.substr(6, 5);

    // 省略第二个参数时取到末尾
    string sub2 = s.substr(6);

    // 翻转字符串
    reverse(s.begin(), s.end());
}

// ==========================================
// 3. 查找与替换
// ==========================================
void string_find() {
    string s = "apple pen apple";

    // 正向查找: 返回起始下标, 找不到返回 string::npos
    size_t pos1 = s.find("apple");
    if (pos1 != string::npos) {
        // pos1 = 0
    }

```

```cpp
    // 逆向查找，返回最后一次出现的位置
    size_t pos2 = s.rfind("apple"); // pos2 = 10

    // 从指定位置开始查找，跳过第一个 apple
    size_t pos3 = s.find("apple", 1); // pos3 = 10

    // 将从下标 0 开始长度 5 的子串替换为 "orange"
    s.replace(0, 5, "orange"); // "orange pen apple"
}

// ==========================================
// 4. 类型转换
// ==========================================
void string_conversions() {
    // 1. 数字转字符串 (C++11)
    int a = 123;
    double d = 3.14;
    string s1 = to_string(a); // "123"
    string s2 = to_string(d); // "3.140000"

    // 2. 字符串转数字
    string num_str = "10086";
    string ll_str = "10000000000000000";

    int num1 = stoi(num_str);       // string 转 int
    long long num2 = stoll(ll_str);// string 转 long long
    double num3 = stod("3.14");     // string 转 double
}

// ==========================================
// 5. Stringstream 分割输入
// ==========================================
// 一行由空格分隔的若干单词或数字，个数未知，且可能有连续多个空格。
// 放入 stringstream 后当作 cin 使用。

void stringstream_magic() {
    string line = "  apple   123  banana  456   ";
    stringstream ss(line);

    string word;
    // 自动跳过连续空格
    while (ss >> word) {
        // 依次输出: "apple", "123", "banana", "456"
```

```
        cout << word << endl;
    }

    // 也可用于类型转换
    string data = "10 20 30";
    stringstream ss2(data);
    int x;
    int sum = 0;
    while (ss2 >> x) {
        sum += x;
    }
}
```

2.11 tuple

```
#include <bits/stdc++.h>
using namespace std;

// ==========================================
// 1. 代替临时结构体，返回多个值
// ==========================================
// 需要同时返回是否有环、环的起点与环的长度时，可用 tuple 代替临时结构体。
tuple<bool, int, int> check_cycle() {
    // 逻辑省略...
    bool has_cycle = true;
    int start_node = 5;
    int length = 3;

    // 用大括号打包返回
    return {has_cycle, start_node, length};
}

void tuple_magic() {
    // ==========================================
    // 2. 结构化绑定 (C++17)
    // ==========================================
    // 无需 std::get<0>(t)，直接按顺序解构：
    auto [is_cyc, st, len] = check_cycle();

    if (is_cyc) {
        cout << "发现环！起点：" << st << "，长度：" << len << "\n";
    }

```

```
    // ========================================
    // 3. 字典序比较
    // ========================================
    // Kruskal 需按边权排序，边为 (w, u, v)。
    // 存入 vector<tuple> 即可直接 sort，无需自定义结构体与比较函数。

    vector<tuple<int, int, int>> edges;
    edges.push_back({10, 1, 2}); // 边权 10，1 连 2
    edges.push_back({5, 2, 3});  // 边权 5，2 连 3
    edges.push_back({10, 2, 4}); // 边权 10，2 连 4

    // sort 按元素依次比较：先比边权，再比起点，最后比终点。
    sort(edges.begin(), edges.end());

    // 遍历排序后的边
    for (auto [w, u, v] : edges) {
        cout << "权值: " << w << " 起点: " << u << " 终点: " << v << "\n";
    }
}

// ========================================
// 4. BFS 状态存储
// ========================================
void tuple_bfs_example() {
    // 走迷宫时状态为 (x, y, keys, steps)，可用 queue<tuple> 存储。
    queue<tuple<int, int, int, int>> q;

    // 初始状态入队：x=0, y=0, keys=0, steps=0
    q.push({0, 0, 0, 0});

    while (!q.empty()) {
        // 取出全部状态
        auto [x, y, keys, steps] = q.front();
        q.pop();

        // 进行上下左右扩展...
    }
}
```

2.12 vector

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp

// ==========================================
// 1. 声明与初始化
// ==========================================
void vector_init() {
    // 基础一维
    vector<int> v1;                    // 空 vector
    vector<int> v2(10);                // 长度为 10，默认全为 0
    vector<int> v3(10, -1);            // 长度为 10，全部初始化为 -1
    vector<int> v4 = {1, 2, 3, 4, 5};  // 列表初始化
    vector<int> v5(v4.begin() + 1, v4.end()); // 截取初始化 (2,3,4,5)

    // 二维及多维
    vector<vector<int>> mat1(5, vector<int>(5, 0)); // 5x5 矩阵，全为 0
    vector<int> adj[100005];           // 邻接表，最常用的存图方式
    vector<vector<pair<int, int>>> graph(100005); // 带权图邻接表
}

// ==========================================
// 2. 容量与内存
// ==========================================
void vector_memory() {
    vector<int> v;

    // reserve 与 resize 的区别
    v.reserve(100000); // 预分配内存，size 仍为 0，避免反复扩容
    v.resize(100000);  // size 变为 10 万，新元素默认填 0

    // 空间释放
    v.clear();         // size 变 0，capacity 通常不释放
    v.shrink_to_fit(); // C++11，将 capacity 缩小到 size
    vector<int>().swap(v); // 彻底释放内存的惯用法
}

// ==========================================
// 3. 增删改
// ==========================================
void vector_modifiers() {
    vector<int> v = {1, 2, 3, 4, 5};

    // 尾部操作
    v.push_back(6);
    v.emplace_back(7); // C++11，原地构造，存结构体时优于 push_back
```

```cpp
    v.pop_back();       // 删除最后一个元素，不返回该元素

    // 插入与删除，复杂度 O(N)
    v.insert(v.begin() + 2, 99); // 在索引 2 前面插入 99
    v.erase(v.begin() + 1);      // 删除索引 1 的元素
    v.erase(v.begin() + 1, v.begin() + 3); // 删除区间 [1, 3)

    // 不关心元素顺序时，可 O(1) 删除指定位置：
    // 与末尾元素交换后 pop_back
    int idx_to_remove = 2;
    swap(v[idx_to_remove], v.back());
    v.pop_back();
}

// ==========================================
// 4. 与 Algorithm 库配合
// ==========================================
void vector_algorithms() {
    vector<int> v = {5, 2, 8, 2, 9, 1};

    // 1. 最值与求和
    int max_val = *max_element(v.begin(), v.end()); // 找最大值
    int min_val = *min_element(v.begin(), v.end()); // 找最小值
    long long sum = accumulate(v.begin(), v.end(), 0LL); // 求和，注意第三
        个参数 0LL 防溢出

    // 2. 排序与翻转
    sort(v.begin(), v.end());              // 升序
    sort(v.rbegin(), v.rend());            // 降序
    reverse(v.begin(), v.end());           // 翻转整个 vector

    // 3. 离散化去重
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());

    // 4. 二分查找 (要求序列有序)
    // lower_bound 找第一个 >= x 的位置
    int pos1 = lower_bound(v.begin(), v.end(), 5) - v.begin();
    // upper_bound 找第一个 > x 的位置
    int pos2 = upper_bound(v.begin(), v.end(), 5) - v.begin();
}

// ==========================================
```

```cpp
// 5. vector<bool>
// ========================================
void vector_bool_warning() {
    // vector<bool> 是位压缩特化，每个 bool 占 1 bit。
    // 省空间，但速度慢，且不能取元素的指针或引用。
    vector<bool> vb(100, false);

    // 需要速度时改用 vector<char> 或 vector<int>
    vector<char> fast_bool(100, 0);
}
```

3 博弈论

3.1 sg 函数

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int sg[N], f[105], k; // f 为允许的步数，需升序；k 为步数种类数
int vis[N];

void get_sg(int n) {
    for (int i = 1; i <= n; i++) {
        // 用时间戳代替 memset，标记一次 O(1)
        for (int j = 1; j <= k && f[j] <= i; j++) {
            vis[sg[i - f[j]]] = i;
        }
        // 求 mex
        for (int j = 0; ; j++) {
            if (vis[j] != i) {
                sg[i] = j;
                break;
            }
        }
    }
}
int main() {
    // 1. 初始化规则数组，必须升序
    k = 3;
    f[1] = 1;
    f[2] = 3;
```

```
    f[3] = 4;
    // sort(f + 1, f + 1 + k); // 规则未升序时需先排序

    // 2. 预处理 SG 函数
    // 设单堆石子最多 10000 个
    get_sg(10000);

    // 3. 多堆石子异或和
    int m;
    cin >> m; // m 堆石子

    int ans = 0;
    for (int i = 1; i <= m; i++) {
        int a;
        cin >> a; // 这一堆有 a 个石子

        // 异或上该堆的 SG 值
        ans ^= sg[a];
    }

    // 4. 异或和非 0 先手必胜，否则后手必胜
    if (ans != 0) {
        cout << "先手必胜！" << endl;
    } else {
        cout << "后手必胜！" << endl;
    }

    return 0;
}
```

4 图论

4.1 dij

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
int n,m,q,k,inf=1e9+7;
struct node{
```

```
    int v,w;
    friend bool operator<(node a,node b){
        return a.w>b.w;
    };
};
void solve(){
    cin>>n>>m;
    vector<vector<node>>f(n+1);
    vector<int>dis(n+1,inf),vis(n+1,0);
    up(i,1,m){
        int u,v,w;
        cin>>u>>v>>w;
        f[u].push_back({v,w});
        f[v].push_back({u,w});
    }
    dis[1]=0;
    priority_queue<node>q;
    q.push({1,0});
    while(!q.empty()){
        int u=q.top().v;
        q.pop();
        if(vis[u])continue;
        vis[u]=1;
        for(auto [v,w]:f[u])if(dis[v]>dis[u]+w){
            dis[v]=dis[u]+w;
            q.push({v,dis[v]});
        }
    }
    up(i,2,n){
        if(dis[i]!=inf)cout<<dis[i]<<'\n';
        else cout<<-1<<'\n';
    }
}
```

4.2 欧拉路径

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
struct node{
    int v,id;
```

```
};
void solve(){
    int n,m;
    cin>>n>>m;
    vector<int>ind(n+1,0),head(n+1,0),vis(m+1,0);
    vector<int>ans;
    vector<vector<node>>f(n+1);
    up(i,1,m){
        int u,v;
        cin>>u>>v;
        f[u].push_back({v,i});
        ind[v]++;
        ind[u]--;
    }
    int rt=1;
    up(i,1,n)if(ind[i]==-1)rt=i;
    sort(ind.begin()+1,ind.end());
    bool ok=1;
    if(ind[1]==0)ok=0;
    if(ind[1]==-1&&ind[n]==1){
        ok=0;
        up(i,2,n-1)ok=(ind[i]!=0);
    }
    if(ok){cout<<"No"<<'\n';return ;}
    up(i,1,n)if(f[i].size()){
        sort(f[i].begin(),f[i].end(),[&](node a,node b){
            return a.v<b.v;
        });
    }
    auto dfs=[&](auto &&self,int u)->void{
        for(int &i=head[u];i<(ll)f[u].size();){
            node e=f[u][i++];
            if(!vis[e.id]){
                vis[e.id]=1;
                self(self,e.v);
            }
        }
        ans.push_back(u);
    };
    dfs(dfs,rt);
    reverse(ans.begin(),ans.end());
    for(auto ele:ans)cout<<ele<<" ";
}
```

```
//输出字典序最小的欧拉路径
```

5 数学

5.1 FFT

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
// FFT: 复数多项式乘法, O(n log n)
// 适用条件: 系数非负且不太大 (一般 |系数| < 1e5、长度 < 1e6 时结果 < 1e16
    , double 够用)
// 如果要精确大数或模意义下的结果, 用 NTT / 任意模数 NTT
const double PI=acos(-1);
using cd=complex<double>;
void fft(vector<cd>&a,bool inv)//inv=true 时做逆变换 (不除 n)
{
    int n=a.size();
    for(int i=1,j=0;i<n;i++){//位反转置换
        int bit=n>>1;
        for(;j&bit;bit>>=1)j^=bit;
        j^=bit;
        if(i<j)swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        double ang=2*PI/len*(inv?-1:1);
        cd w(cos(ang),sin(ang));//len 次单位根
        for(int i=0;i<n;i+=len){
            cd wn(1);
            for(int k=0;k<len/2;k++){
                cd u=a[i+k], v=a[i+k+len/2]*wn;
                a[i+k]=u+v, a[i+k+len/2]=u-v;
                wn*=w;
            }
        }
    }
}
vector<ll> conv(vector<ll> a,vector<ll> b)//多项式乘法, 结果四舍五入回整数
{
```

```
    if(a.empty()||b.empty())return {};
    int need=a.size()+b.size()-1, n=1;
    while(n<need)n<<=1;
    vector<cd> fa(a.begin(),a.end()), fb(b.begin(),b.end());
    fa.resize(n), fb.resize(n);
    fft(fa,0), fft(fb,0);
    up(i,0,n-1)fa[i]*=fb[i];
    fft(fa,1);
    vector<ll> c(need);
    up(i,0,need-1)c[i]=llround(fa[i].real()/n);
    return c;
}
void solve(){
    int n,m;
    cin>>n>>m;
    vector<ll>a(n+1),b(m+1);
    up(i,0,n)cin>>a[i];
    up(i,0,m)cin>>b[i];
    auto c=conv(a,b);
    up(i,0,n+m)cout<<c[i]<<" \n"[i==n+m];
}
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
/*
2 1
1 2 1
1 1
// 1 3 3 1
*/
```

5.2 NTT

```
#include <bits/stdc++.h>
using namespace std;
```

```
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
// NTT: 模 998244353 的多项式乘法, O(n log n)
// 998244353 = 119*2^23+1, 原根 3, 所以长度必须是 2 的幂且不超过 2^23
const int MOD=998244353, G=3;
int ksm(int a,int b,int p){int r=1%p;for(;b;b>>=1,a=a*a%p)if(b&1)r=r*a%p;
    return r;}
void ntt(vector<int>&a,bool inv)//inv=true 时做逆变换 (不除 n)
{
    int n=a.size();
    for(int i=1,j=0;i<n;i++){//位反转置换
        int bit=n>>1;
        for(;j&bit;bit>>=1)j^=bit;
        j^=bit;
        if(i<j)swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        int w=ksm(G,(MOD-1)/len,MOD);//len 次单位根
        if(inv)w=ksm(w,MOD-2,MOD);
        for(int i=0;i<n;i+=len){
            int wn=1;
            for(int k=0;k<len/2;k++){
                int u=a[i+k], v=a[i+k+len/2]*wn%MOD;
                a[i+k]=(u+v)%MOD;
                a[i+k+len/2]=(u-v+MOD)%MOD;
                wn=wn*w%MOD;
            }
        }
    }
}
vector<int> conv(vector<int> a,vector<int> b)//多项式乘法
{
    if(a.empty()||b.empty())return {};
    int need=a.size()+b.size()-1, n=1;
    while(n<need)n<<=1;
    a.resize(n), b.resize(n);
    ntt(a,0), ntt(b,0);
    up(i,0,n-1)a[i]=a[i]*b[i]%MOD;
    ntt(a,1);
    int inv_n=ksm(n,MOD-2,MOD);
    up(i,0,need-1)a[i]=a[i]*inv_n%MOD;
```

```
    a.resize(need);
    return a;
}
void solve(){
    int n,m;
    cin>>n>>m;
    vector<int>a(n+1),b(m+1);
    up(i,0,n)cin>>a[i];
    up(i,0,m)cin>>b[i];
    auto c=conv(a,b);
    up(i,0,n+m)cout<<c[i]<<" \n"[i==n+m];
}
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
/*
2 1
1 2 1
1 1
// 1 3 3 1

长度是 2 的幂、且不超过 2^23 = 8388608 才能直接用。
*/
```

5.3 exgcd

```
/**
 * 扩展欧几里得算法 (EXGCD)
 * 求解 ax + by = gcd(a, b) 的一组特解 (x, y), 并返回 gcd(a, b)
 */
int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
```

```
    }
    int d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

5.4 三分

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
// 三分：求单峰函数的最值。凸函数（中间低两边高）用 *_min，凹函数（中间高两边低）用 *_max
// 函数用 lambda 传进来，所以实数、整数、二维（三分套三分）都能直接用
//   例：auto f = [&](long double x){ return (x - 2) * (x - 2) + 3; };
//       long double x = ternary_min(-10, 10, f);   // 极小值点

// ---------- 实数域 ----------
// 在 [l, r] 上找极小值点，返回 x（要函数值就再求一次 f(x)）
template <class F>
double ternary_min(double l, double r, F f)
{
    up(it, 1, 100) {//100 次收缩后区间缩小到 (2/3)^100，早就到 double 精度极限
        double m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
        if (f(m1) < f(m2)) r = m2;//极小值在 [l, m2] 里
        else l = m1;              //极小值在 [m1, r] 里
    }
    return (l + r) / 2;
}
// 找极大值点：对 -f 求极小
template <class F>
double ternary_max(double l, double r, F f)
{
    return ternary_min(l, r, [&](double x) { return -f(x); });
}

// ---------- 整数域 ----------
// 先收缩到长度 <= 2，再暴力比较剩下的几个点（整数下这样最稳，不会因为取整漏掉答案）
```

```cpp
template <class F>
int ternary_min_int(int l, int r, F f)
{
    while (r - l > 2) {
        int m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
        if (f(m1) < f(m2)) r = m2;
        else l = m1;
    }
    int best = l;
    up(i, l + 1, r)if (f(i) < f(best))best = i;
    return best;
}
// 极大值点: 同样对 -f 求极小
template <class F>
int ternary_max_int(int l, int r, F f)
{
    return ternary_min_int(l, r, [&](int x) { return -f(x); });
}

void solve(){
    auto f = [](double x) { return (x - 2) * (x - 2) + 3; };//凸: 最小值在 x=2
    auto g = [](double x) { return -(x + 1) * (x + 1) + 5; };//凹: 最大值在 x=-1
    auto h = [](int x) { return (x - 5) * (x - 5); };//凸: 最小值在 x=5
    auto t = [](int x) { return -(x - 7) * (x - 7) + 9; };//凹: 最大值在 x=7

    double x1 = ternary_min(-10.0, 10.0, f);
    double x2 = ternary_max(-10.0, 10.0, g);
    int x3 = ternary_min_int(0, 20, h);
    int x4 = ternary_max_int(0, 20, t);

    cout << fixed << setprecision(6);
    cout << "凸(实) 极值点 x = " << x1 << ", 值 = " << f(x1) << '\n';
    cout << "凹(实) 极值点 x = " << x2 << ", 值 = " << g(x2) << '\n';
    cout << "凸(整) 极值点 x = " << x3 << ", 值 = " << h(x3) << '\n';
    cout << "凹(整) 极值点 x = " << x4 << ", 值 = " << t(x4) << '\n';
}

signed main()
{
    ios::sync_with_stdio(0);
```

```
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
/*
凸(实) 极值点 x = 2.000000, 值 = 3.000000
凹(实) 极值点 x = -1.000000, 值 = 5.000000
凸(整) 极值点 x = 5, 值 = 0
凹(整) 极值点 x = 7, 值 = 9

拓展:
1. 二维(三分套三分): 外层三分 y, 内层对每个 y 三分 x 求最优, 注意外层函数值就是内层的最优值;
2. 复杂度: 实数域 O(100) 次求值, 整数域 O(log n) 次求值;
3. 函数必须严格单峰(或有平顶也行), 多峰的题要换思路。
*/
```

5.5 中国剩余定理

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e5+5,mod=998244353,inf=1e9+7;
int n,L;
int a[N],m[N];//a:余数 m:模数 L:合并后的总模数
int exgcd(int a,int b,int &x,int &y)
{
    if(b==0){x=1,y=0;return a;}
    int d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
}
int norm(int x,int p){return (x%p+p)%p;}//归一到[0,p)
int addm(int a,int b,int p){return a>=p-b?a-(p-b):a+b;}//要求a,b<p
```

```
20 int mul(int a,int b,int p){return (__int128)norm(a,p)*norm(b,p)%p;}//防爆乘(先归一化，exgcd出来的系数可能是负的)
21 int crt()//朴素CRT，要求模数两两互质
22 {
23     int M=1,res=0;
24     up(i,1,n)M*=m[i];
25     up(i,1,n){
26         int Mi=M/m[i],x,y;
27         exgcd(Mi,m[i],x,y);//Mi*x+m[i]*y=1，x即逆元
28         res=addm(res,mul(mul(norm(a[i],M),Mi,M),norm(x,m[i]),M),M);
29     }
30     L=M;
31     return res;//x=sum(a[i]*Mi*inv(Mi))
32 }
33 int excrt()//模数任意，无解返回-1，通解x+k*L
34 {
35     int x=0,M=1;
36     up(i,1,n){
37         int t,s;
38         int g=exgcd(M,m[i],t,s);//M*t+m[i]*s=g
39         int c=norm(a[i]-x,m[i]);
40         if(c%g)return -1;//裴蜀定理：g不整除c则无解
41         int M2=M/g*m[i];//lcm，先除后乘
42         x=addm(norm(x,M2),mul(M,mul(t,c/g,m[i]/g),M2),M2);
43         M=M2;
44     }
45     L=M;
46     return x;
47 }
48 void solve(){
49     cin>>n;
50     up(i,1,n)cin>>a[i]>>m[i];
51     int x=excrt();//互质时直接crt()
52     if(x==-1)cout<<-1<<'\n';
53     else cout<<x<<'\n';
54 }
55
56 signed main()
57 {
58     ios::sync_with_stdio(0);
59     cin.tie(0), cout.tie(0);
60     int t = 1;
61 //  cin >> t;
```

```
62      while (t--){
63          solve();
64      }
65  //  system("x-www-browser https://ys.mihoyo.com/");
66      return 0;
67  }
68  /*
69  3
70  2 3
71  3 5
72  2 7
73  // 23
74
75  3
76  2 4
77  3 6
78  2 5
79  // -1
80
81  P1495 曹冲养猪 P4777 扩展中国剩余定理 P3868 猜数字 P4774 屠龙勇士 P2480 古代猪文
82  */
```

5.6 中国剩余定理（推导笔记）

1. 问题模型与结论

解同余方程组 $x \equiv a_i \pmod{m_i}$ $(i = 1, \ldots, n)$。

- **CRT**：若 $m_1, \ldots, m_n$ **两两互质**，令 $M = \prod m_i$，则必有解，且在模 M 下唯一，即 $[0, M)$ 内恰有一个解。本质是把" 模 M" 拆成若干个" 模 m_i" 独立求解再拼回。
- **通解**：$x = x_0 + kM$；若要求答案 ≥ 1，则最小正整数解为

$$\big((x_0 - 1) \bmod M\big) + 1$$

- **有解判据**（模数不互质时）：$\forall i, j:\ a_i \equiv a_j \pmod{\gcd(m_i, m_j)}$。

2. 模数互质：构造法

记 $M_i = \dfrac{M}{m_i}$。互质保证 $\gcd(M_i, m_i) = 1$，故逆元 $t_i \equiv M_i^{-1} \pmod{m_i}$ 存在（用 exgcd 解 $M_i t_i + m_i s_i = 1$，**不可用费马小定理**）：

$$\boxed{x \equiv \sum_{i=1}^{n} a_i M_i t_i \pmod{M}}$$

正确性：放到模 m_j 下看，$i \neq j$ 时 $m_j \mid M_i$，该项为 0；$i = j$ 时 $M_j t_j \equiv 1$，该项为 a_j。故 $x \equiv a_j \pmod{m_j}$ 对每个 j 成立。

唯一性：x, y 均为解 $\Rightarrow m_i \mid (x-y)$ 对每个 i；两两互质 $\Rightarrow M \mid (x-y)$，即 $x \equiv y \pmod{M}$。

复杂度 $O(n \log M)$。

---

3. 模数不互质：EXCRT 逐方程合并

此时 $\gcd(M_i, m_i) \neq 1$，逆元不存在，构造法失效。改为把两个方程并成一个。

设 $x = a_1 + m_1 k$ 代入第二式，令 $g = \gcd(m_1, m_2)$：

$$m_1 k \equiv a_2 - a_1 \pmod{m_2}$$

由裴蜀定理，有解 $\iff g \mid (a_2 - a_1)$；由 exgcd 解 $m_1 t + m_2 s = g$ 得

$$k \equiv \frac{a_2 - a_1}{g} \cdot t \pmod{\frac{m_2}{g}}$$

回代即得**两个方程等价于一个方程**（模数被约掉了因子 g，这是 EXCRT 快的原因）：

$$\boxed{x \equiv a' \pmod{\frac{m_1}{g} m_2}, \qquad a' = a_1 + m_1 k_0}$$

迭代：维护 $x \equiv x_0 \pmod{M}$（初始 $0, 1$），每次并入 $x \equiv a_i \pmod{m_i}$：

$$g = \gcd(M, m_i), \qquad x \equiv x_0 + Mt \pmod{\frac{M}{g} m_i}$$

其中 t 由 $Mt \equiv a_i - x_0 \pmod{m_i}$ 解出，即 $t \equiv \dfrac{a_i - x_0}{g} \cdot (\text{exgcd 的解}) \pmod{\dfrac{m_i}{g}}$。每步都是充要变形，$n$ 次后得到的单方程与原方程组同解，最终 $M = \operatorname{lcm}(m_1, \ldots, m_n)$。

复杂度 $O(n \log \max m_i)$。

---

4. 实现要点

- 常数 $c = a_i - x_0$ 与特解 t 都先归一到 $[0, \cdot)$。由 $t \leq \frac{m_i}{g} - 1$ 得 $x_0 + Mt < M + (\operatorname{lcm} - M) = \operatorname{lcm}$，即**中间量不超过最终模数**，不会额外溢出；
- lcm 必须**先除后乘**：$\dfrac{M}{g} \cdot m_i$；

- 板子只到 `long long`（$\mathrm{lcm} \le 9.2 \times 10^{18}$）；再大要换 `__int128` 版本，且那时 $a \times b$ 不能再用 `__int128` 中转，须改用龟速乘；
- 防爆乘里**两个乘数都要先归一化**：exgcd 给出的系数可能是负的，负系数直接取模会得到负数，破坏后续" 两数均 $< p$" 的加法前提；
- x_0 已归一到 $[0, \mathrm{lcm})$，故可用 -1 作无解哨兵；
- 余数可能为负或 $\ge m_i$，先 $\big((a_i \bmod m_i) + m_i\big) \bmod m_i$。

---

5. 常考公式速查

用途	公式
裴蜀定理	$ax + by = c$ 有整数解 $\iff \gcd(a,b) \mid c$
一元一次同余方程	$ax \equiv b \pmod m$ 有解 $\iff \gcd(a,m) \mid b$，解为 $x \equiv \frac{b}{g}t \pmod{\frac{m}{g}}$（$g = \gcd(a,m)$）
互质时构造解	$x \equiv \sum a_i M_i t_i \pmod M$，$M_i = \frac{M}{m_i}$，$t_i \equiv M_i^{-1} \pmod{m_i}$
两方程合并	$x \equiv a_1 + m_1 k_0 \pmod{\frac{m_1}{g}m_2}$，$g = \gcd(m_1, m_2)$
最终模数	$M = \mathrm{lcm}(m_1, \ldots, m_n) = \frac{m_1}{g} \cdot m_2$
通解 / 第 k 小非负解	$x = x_0 + (k-1)M$
最小正整数解	$\big((x_0 - 1) \bmod M\big) + 1$
有解判据	$\forall i, j:\ a_i \equiv a_j \pmod{\gcd(m_i, m_j)}$
解不唯一时解的个数	$ax \equiv b \pmod m$ 在 $[0, m)$ 内恰有 $\gcd(a,m)$ 个解
复杂度	CRT $O(n \log M)$，EXCRT $O(n \log \max m_i)$

---

6. 变形与陷阱

- **系数不为一**：$a_i x \equiv b_i \pmod{m_i}$ 先约化为 $x \equiv x_i \pmod{\frac{m_i}{\gcd(a_i, m_i)}}$（$\gcd(a_i, m_i) \nmid b_i$ 时无解），再上 EXCRT；
- **exLucas**：求 $\binom{n}{m} \bmod P$（$P = \prod p_i^{k_i}$ 为合数），分别求 $\bmod p_i^{k_i}$ 再用 CRT 合并；
- **模合数高斯消元 / BSGS**：同样先按质因子拆分，再 CRT 合并；
- **带下界**（如 P4774 屠龙勇士）：答案取 $x_0 + kM$ 中满足下界的**最小**者，不能直接输出 x_0；
- **易错**：模数非素数不能费马求逆元；漏判无解；要求正整数解；合并顺序不影响结果，但先合并小模数不易溢出。

例题 ：P1495 曹冲养猪（互质构造）· P4777 EXCRT 模板 · P3868 猜数字（防溢出）· P4774 屠龙勇士（下界）· P2480 古代猪文（Lucas + CRT）。

5.7 任意模数 NTT

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
// 任意模数 NTT：取三个 NTT 友好素数分别卷积，再用 CRT 拼回真实系数
// 三个素数乘积约 4.7e26；系数 < 1e9、n <= 1e6 时真实系数 < 1e24，能精确还原
const int P1=998244353, P2=1004535809, P3=469762049, G=3;
int MOD;                          // 目标模数，输入后设置
int ksm(int a,int b,int p){int r=1%p;for(;b;b>>=1,a=a*a%p)if(b&1)r=r*a%p;return r;}
void ntt(vector<int>&a,int p,bool inv)
{
    int n=a.size();
    for(int i=1,j=0;i<n;i++){
        int bit=n>>1;
        for(;j&bit;bit>>=1)j^=bit;
        j^=bit;
        if(i<j)swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        int w=ksm(G,(p-1)/len,p);
        if(inv)w=ksm(w,p-2,p);
        for(int i=0;i<n;i+=len){
            int wn=1;
            for(int k=0;k<len/2;k++){
                int u=a[i+k], v=a[i+k+len/2]*wn%p;
                a[i+k]=(u+v)%p;
                a[i+k+len/2]=(u-v+p)%p;
                wn=wn*w%p;
            }
        }
    }
}
vector<int> conv_mod(vector<int> a,vector<int> b,int p)//在模 p 下卷积
{
```

```
37      int need=a.size()+b.size()-1, n=1;
38      while(n<need)n<<=1;
39      a.resize(n), b.resize(n);
40      ntt(a,p,0), ntt(b,p,0);
41      up(i,0,n-1)a[i]=a[i]*b[i]%p;
42      ntt(a,p,1);
43      int inv_n=ksm(n,p-2,p);
44      up(i,0,need-1)a[i]=a[i]*inv_n%p;
45      a.resize(need);
46      return a;
47  }
48  int crt(int r1,int r2,int r3)//三个余数 -> 真实系数 mod MOD
49  {
50      int t1=(r2-r1)%P2*ksm(P1%P2,P2-2,P2)%P2;
51      t1=(t1+P2)%P2;
52      int cur=(r1+P1%P3*(t1%P3))%P3;
53      int t2=(r3-cur+P3)%P3*ksm(P1%P3*(P2%P3)%P3,P3-2,P3)%P3;
54      int ans=r1%MOD;
55      ans=(ans+P1%MOD*(t1%MOD))%MOD;
56      ans=(ans+(P1%MOD)*(P2%MOD)%MOD*(t2%MOD))%MOD;
57      return ans;
58  }
59  vector<int> mtt(vector<int> a,vector<int> b)//任意模数卷积
60  {
61      if(a.empty()||b.empty())return {};
62      auto c1=conv_mod(a,b,P1), c2=conv_mod(a,b,P2), c3=conv_mod(a,b,P3);
63      int need=a.size()+b.size()-1;
64      up(i,0,need-1)c1[i]=crt(c1[i],c2[i],c3[i]);
65      return c1;
66  }
67  void solve(){
68      int n,m;
69      cin>>n>>m>>MOD;
70      vector<int>a(n+1),b(m+1);
71      up(i,0,n)cin>>a[i];
72      up(i,0,m)cin>>b[i];
73      auto c=mtt(a,b);
74      up(i,0,n+m)cout<<c[i]<<" \n"[i==n+m];
75  }
76  signed main()
77  {
78      ios::sync_with_stdio(0);
79      cin.tie(0), cout.tie(0);
```

```
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
    return 0;
}
/*
2 1 1000000007
1 2 1
1 1
// 1 3 3 1
*/
```

5.8 前缀基

```
#include <iostream>
#include <vector>

using ll = long long;
using namespace std;
const int MOD = 998244353;

ll qpow(ll a, ll b) {
    ll res = 1;
    a %= MOD;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

/**
 * 线性批量求逆元（前缀积法）
 * 传入需要求逆元的数组 a (1-indexed)，将结果存入 inv_a (1-indexed)
 * 时间复杂度：O(n + log(MOD))
 */
void linear_inv(const std::vector<ll>& a, std::vector<ll>& inv_a) {
    int n = a.size() - 1;
    if (n <= 0) return;
    std::vector<ll> pre(n + 1, 1), inv_pre(n + 1, 1);
```

```
    for (int i = 1; i <= n; i++) {
        pre[i] = pre[i - 1] * a[i] % MOD;
    }
    inv_pre[n] = qpow(pre[n], MOD - 2);
    for (int i = n; i >= 1; i--) {
        inv_pre[i - 1] = inv_pre[i] * a[i] % MOD;
    }
    for (int i = 1; i <= n; i++) {
        inv_a[i] = inv_pre[i] * pre[i - 1] % MOD;
    }
}
```

5.9 前缀线性基

```
#include<iostream>
#include<stack>
#include<string>
#include<set>
#include<algorithm>
#include<queue>
#include<map>
#include<vector>
#include<cmath>
#include<cstring>
#include<cstdio>
#include<random>
#include<cmath>
#include <functional>
#include<cctype>
#include<bitset>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
//#define int long long
mt19937_64 gen(12345);
const int N=5e5+5;
int n,q,cnt,root[N],last[N];
int ls[N<<6],rs[N<<6],w[N<<6],a[N];
int update(int lu,int l,int r,int x){
    int u=++cnt;
    ls[u]=ls[lu],rs[u]=rs[lu],w[u]=w[lu];w[u]++;
    if(l==r)return u;
```

```
30      int mid=(l+r)>>1;
31      if(x<=mid)ls[u]=update(ls[lu],l,mid,x);
32      else rs[u]=update(rs[lu],mid+1,r,x);
33      return u;
34  }
35  int query(int u1,int u2,int l,int r,int x){
36      if(l>=x)return 0;
37      if(r<x)return w[u2]-w[u1];
38      int mid=(l+r)>>1;
39      return query(ls[u1],ls[u2],l,mid,x)+query(rs[u1],rs[u2],mid+1,r,x);
40  }
41  void solve(){
42      cnt=0;
43      cin>>n>>q;
44      vector<vector<int>>f(n+1);
45      vector<vector<int>>basis(n+1,vector<int>(20,0));
46      vector<vector<int>>pos(n+1,vector<int>(20,0));
47      map<int,int>mp;
48      up(i,1,n){cin>>a[i];last[a[i]]=0;}
49      up(i,1,n-1){
50          int u,v;cin>>u>>v;
51          f[u].push_back(v);
52          f[v].push_back(u);
53      }
54      int tot=0;
55      vector<int>L(n+1,0),R(n+1,0);
56      auto insert=[&](int p,int x)->void{
57          basis[p]=basis[p-1];
58          pos[p]=pos[p-1];
59          int tmp=p;
60          down(i,20,0)if((x>>i)&1){
61              if(!basis[p][i]){
62                  basis[p][i]=x;
63                  pos[p][i]=tmp;
64                  return ;
65              }
66              if(tmp>pos[p][i]){
67                  swap(tmp,pos[p][i]);
68                  swap(x,basis[p][i]);
69              }
70              x^=basis[p][i];
71          }
72      };
```

```
73      auto dfs=[&](auto &&self,int u,int fa)->void{
74          L[u]=++tot;
75          insert(tot,a[u]);
76          if(last[mp[a[u]]])root[tot]=update(root[tot-1],0,n,last[mp[a[u]]]);
77          else root[tot]=update(root[tot-1],0,n,0);
78          last[mp[a[u]]]=tot;
79          for(auto v:f[u])if(v!=fa){
80              self(self,v,u);
81          }
82          R[u]=tot;
83      };
84      dfs(dfs,1,-1);
85      int lastans=0;
86      while(q--){
87          int u;
88          cin>>u;
89          u=((lastans^u)%n+1);
90  //      cout<<u<<'\n';
91          int l=L[u],r=R[u];
92          int k=query(root[l-1],root[r],0,n,1);
93  //      cout<<k<<'\n';
94          lastans=k;
95          down(i,31,0)if((k>>i)&1){
96              if(basis[r][i]&&pos[r][i]>=l){
97                  k^=basis[r][i];
98              }
99          }
100         if(k==0)cout<<"your shine your be!\n";
101         else cout << "-1\n";
102     }
103 }
104
105 signed main()
106 {
107     ios::sync_with_stdio(0);
108     cin.tie(0), cout.tie(0);
109     int t = 1;
110     cin >> t;
111     while (t--){
112         solve();
113     }
114 //  system("x-www-browser https://ys.mihoyo.com/");
115     return 0;
```

```
}
/*
1
5 3
4 3 2 0 0
1 4
1 3
2 1
1 5
4
1
5

*/
```

5.10 卢卡斯定理

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e6+5,mod=998244353,inf=1e9+7;
int n,m,p,fac[N];//fac:阶乘，Lucas用，要求p<N
int ksm(int a,int b,int p)
{
    int res=1;
    while(b){
        if(b&1)res=res*a%p;
        a=a*a%p;
        b>>=1;
    }
    return res;
}
int exgcd(int a,int b,int &x,int &y)
{
    if(b==0){x=1,y=0;return a;}
    int d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
}
```

```
int norm(int x,int p){return (x%p+p)%p;}
int inv(int a,int p){int x,y;exgcd(a,p,x,y);return norm(x,p);}
int isprime(int x)
{
    if(x<2)return 0;
    for(int i=2;i*i<=x;i++)if(x%i==0)return 0;
    return 1;
}
int C(int n,int m,int p)//n,m<p, 用阶乘+费马小定理
{
    if(m>n)return 0;
    return fac[n]*ksm(fac[m],p-2,p)%p*ksm(fac[n-m],p-2,p)%p;
}
int lucas(int n,int m,int p)//p必须是素数, 用前需init(p)
{
    if(m==0)return 1;
    return C(n%p,m%p,p)*lucas(n/p,m/p,p)%p;
}
int facpk(int n,int p,int pk)//n!去掉所有p因子后mod pk
{
    if(n==0)return 1;
    int res=1;
    up(i,1,pk)if(i%p)res=res*i%pk;//一个完整周期的积
    res=ksm(res,n/pk,pk);
    up(i,1,n%pk)if(i%p)res=res*i%pk;//零头
    return res*facpk(n/p,p,pk)%pk;
}
int Cpk(int n,int m,int p,int pk)//C(n,m) mod pk
{
    if(n<m)return 0;
    int e=0;
    for(int i=n;i;i/=p)e+=i/p;
    for(int i=m;i;i/=p)e-=i/p;
    for(int i=n-m;i;i/=p)e-=i/p;//e=v_p(n!)-v_p(m!)-v_p((n-m)!)
    int a=facpk(n,p,pk),b=facpk(m,p,pk),c=facpk(n-m,p,pk);
    return a*inv(b,pk)%pk*inv(c,pk)%pk*ksm(p,e,pk)%pk;
}
int exlucas(int n,int m,int mod)//mod任意, 要求分解出的pk不大
{
    if(n<m)return 0;
    int res=0,M=1,x=mod;//已合并的答案 res mod M
    for(int i=2;i*i<=x;i++){
        if(x%i)continue;
```

```
70          int pk=1;
71          while(x%i==0)x/=i,pk*=i;
72          int t=norm(Cpk(n,m,i,pk)-res,pk)*inv(M%pk,pk)%pk;
73          res+=M*t,M*=pk,res%=M;
74      }
75      if(x>1){//剩下一个大质因子
76          int pk=x;
77          int t=norm(Cpk(n,m,x,pk)-res,pk)*inv(M%pk,pk)%pk;
78          res+=M*t,M*=pk,res%=M;
79      }
80      return res;
81  }
82  void solve(){
83      cin>>n>>m>>p;
84      if(isprime(p)&&p<N){//素数且不太大，Lucas更快
85          fac[0]=1;
86          up(i,1,p-1)fac[i]=fac[i-1]*i%p;
87          cout<<lucas(n,m,p)<<'\n';
88      }
89      else cout<<exlucas(n,m,p)<<'\n';
90  }
91
92  signed main()
93  {
94      ios::sync_with_stdio(0);
95      cin.tie(0), cout.tie(0);
96      int t = 1;
97      cin >> t;
98      while (t--){
99          solve();
100     }
101 //  system("x-www-browser https://ys.mihoyo.com/");
102     return 0;
103 }
104 /*
105 2
106 5 3 7
107 6 4 9
108 // 3
109 // 6
110
111 P3807 卢卡斯定理 P4720 扩展卢卡斯定理 P2480 古代猪文
112 */
```

5.11 卢卡斯定理（推导笔记）

1. 问题模型

求 $\binom{n}{m} \bmod p$。p 为**素数**时用 Lucas 定理，p 为**合数**时用扩展 Lucas。

---

2. Lucas 定理与证明

p 为素数，则

$$\boxed{\binom{n}{m} \equiv \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor}\binom{n \bmod p}{m \bmod p} \pmod p}$$

证明（生成函数）：在模 p 意义下

$$(1+x)^p = \sum_{i=0}^{p}\binom{p}{i}x^i \equiv 1 + x^p \pmod p$$

因为 $0 < i < p$ 时 $p \mid \binom{p}{i} = \dfrac{p!}{i!\,(p-i)!}$：分子含因子 p，而分母的 $i!\,(p-i)!$ 与 p 互质。

写 $n = sp + r$，其中 $s = \lfloor n/p \rfloor$，$r = n \bmod p$，则

$$(1+x)^n = \big((1+x)^p\big)^s(1+x)^r \equiv (1+x^p)^s(1+x)^r \pmod p$$

比较两端 x^m 的系数。右端 $= \sum_{i,j}\binom{s}{i}\binom{r}{j}x^{ip+j}$。再把 m 写成 $m = s'p + r'$，由于 $0 \le j \le r < p$，方程 $ip + j = m$ 的解唯一：$i = s'$，$j = r'$（若 $r' > r$ 则无解，此时 $\binom{r}{r'} = 0$）。于是

$$\binom{n}{m} \equiv \binom{s}{s'}\binom{r}{r'} = \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor}\binom{n \bmod p}{m \bmod p} \pmod p$$

递归形式：把 n, m 写成 p 进制 $n = \sum n_i p^i$，$m = \sum m_i p^i$，反复代入即得

$$\binom{n}{m} \equiv \prod_{i \ge 0}\binom{n_i}{m_i} \pmod p$$

所以**只要有某一位 $m_i > n_i$（此时 $\binom{n_i}{m_i} = 0$），答案就是 0**。

实现：p 不大时预处理 $0! \sim (p-1)!$，单个组合数用费马小定理求逆，每次询问 $O(\log_p n)$；预处理 $O(p)$。

---

3. Kummer 定理与常用推论

$$\boxed{v_p\left(\binom{n}{m}\right) = p \text{ 进制下计算} m + (n-m) \text{ 的进位次数}}$$

推论（常考）：$\binom{n}{m}$ 对一切 $0 < m < n$ 都不被 p 整除 $\iff$ n 是 p 的幂。（无进位 $\iff$ m 的每一位都不超过 n 的对应位；要让每个 $0 < m < n$ 都做不到，n 必须只有一位非零。）

---

4. 扩展 Lucas：模数合数

设 $P=\prod_i p_i^{k_i}$。**思路**：先在每个素数幂模 $p_i^{k_i}$ 下分别求出 $\binom{n}{m}$，这些模数两两互质，最后用 CRT 合并（见 CRT 笔记）。

4.1 困难所在 $\binom{n}{m}=\dfrac{n!}{m!\,(n-m)!}$，但 $p\mid n!$ 时阶乘在模 p^k 下**不可逆**，不能直接求逆。

4.2 把 p 的幂次单独提出来 记 $g(N)=\prod_{1\le j\le N,\ p\nmid j} j$，即 $[1,N]$ 中所有非 p 倍数的乘积。把 $1\sim n$ 分成"p 的倍数" 与" 非 p 的倍数" 两类：

$$n!=\underbrace{g(n)}_{\text{非}p\text{ 倍数}}\cdot\underbrace{p^{\lfloor n/p\rfloor}\left\lfloor\frac{n}{p}\right\rfloor!}_{p\text{ 的倍数，各提一个}p}$$

对 $\lfloor n/p\rfloor!$ 继续递归，即得

$$n!=p^{\,e}\cdot u(n),\qquad e=v_p(n!)=\sum_{i\ge1}\left\lfloor\frac{n}{p^i}\right\rfloor,\qquad u(n)=\prod_{i\ge0} g\!\left(\left\lfloor\frac{n}{p^i}\right\rfloor\right)$$

4.3 g 的快速计算（周期性） 模 p^k 下，与 p 互质的余数类以 p^k 为周期。记 $f=\prod_{1\le i\le p^k,\ p\nmid i} i$，则

$$g(N)\equiv f^{\lfloor N/p^k\rfloor}\cdot\prod_{\substack{1\le i\le N \bmod p^k\\ p\nmid i}} i\pmod{p^k}$$

即"$f^{\text{整周期数}}\times$ 零头"，$O(p^k)$ 次乘法——这就是要求 p^k 不能太大的原因。

4.4 组合数公式 $u(\cdot)$ 的每一项都与 p 互质，故在模 p^k 下可逆：

$$\boxed{\binom{n}{m}\equiv p^{\,e}\cdot u(n)\cdot u(m)^{-1}\cdot u(n-m)^{-1}\pmod{p^k},\qquad e=v_p(n!)-v_p(m!)-v_p((n-m)!)}$$

注意 $e\ge0$；若 $e\ge k$，则 $p^e\equiv0\pmod{p^k}$，答案为 0。

4.5 合并 各 $p_i^{k_i}$ 两两互质，用 CRT 逐个合并即可（约定 $m>n$ 时 $\binom{n}{m}=0$）。

5. 常考公式速查

用途	公式
Lucas 定理	$\binom{n}{m} \equiv \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor}\binom{n \bmod p}{m \bmod p} \pmod p$（$p$ 素数）
p 进制乘积形式	$\binom{n}{m} \equiv \prod_i \binom{n_i}{m_i} \pmod p$，某位 $m_i > n_i$ 则为 0
Kummer 定理	$v_p\binom{n}{m} = p$ 进制下 $m+(n-m)$ 的进位次数
整除推论	$p \nmid \binom{n}{m}\ (\forall\, 0<m<n) \iff n = p^k$
阶乘分解	$n! = p^{\sum_i \lfloor n/p^i \rfloor} \cdot u(n)$，$u(n) = \prod_i g(\lfloor n/p^i \rfloor)$
g 的周期	$g(N) \equiv f^{\lfloor N/p^k \rfloor} \cdot (\text{零头}) \pmod{p^k}$，$f = \prod_{i \le p^k,\, p \nmid i} i$
exLucas 单项	$\binom{n}{m} \equiv p^e u(n) u(m)^{-1} u(n-m)^{-1} \pmod{p^k}$
exLucas 合并	$P = \prod p_i^{k_i}$，各 $p_i^{k_i}$ 上分别求值后 CRT 合并
单点组合数（$n < p$）	$\binom{n}{m} \equiv n! \cdot (m!)^{-1} \cdot ((n-m)!)^{-1} \pmod p$，逆元用费马小定理
复杂度	Lucas $O(p + \log_p n)$；exLucas $O\big(\sum_i p_i^{k_i} \log n\big)$

6. 陷阱

- p **必须是素数**才能用 Lucas。p 为合数时既不能用 Lucas，也不能用费马小定理求逆元（要用 exgcd）；
- $m > n$ 时返回 0；$m = 0$ 时返回 1，递归边界别写错；
- 预处理阶乘只需到 $p-1$（不是 n），这正是 Lucas 能处理 $n \le 10^{18}$ 的原因；
- 若 p 很大（如 10^9+7）而 $n < p$，则**不必用 Lucas**，直接预处理到 n 用费马求逆即可；
- exLucas 的复杂度含 p^k 因子，只适合模数较小（P4720 中 $p \le 10^6$）的场合；
- exLucas 里 $e \ge k$ 时答案直接为 0，不必再算 u。

例题 ：P3807 【模板】卢卡斯定理 · P4720 【模板】扩展卢卡斯 · P2480 古代猪文（Lucas + CRT）· P6669 多项式（Lucas 数位 DP）。

5.12 多项式卷积（推导笔记）

1. 卷积与点值表示

两个多项式的乘积

$$c_k = \sum_{i+j=k} a_i b_j$$

直接算是 $O(n^2)$。但把多项式换成**点值表示**（在若干点处的取值），乘法就变成逐点相乘：

$$\text{系数表示} \xrightarrow{\text{求值}} \text{点值表示} \xrightarrow{\text{逐点相乘}} \text{点值表示} \xrightarrow{\text{插值}} \text{系数表示}$$

只要取 $n \geq \deg a + \deg b + 1$ 个点，逐点相乘后插值就能得到准确的乘积。若能在 $O(n \log n)$ 内完成求值与插值，总复杂度就是 $O(n \log n)$。选点要选得「有结构」，这就是单位根。

2. DFT 与单位根

令 $\omega_n = e^{2\pi i/n}$（n 次本原单位根）。**离散傅里叶变换（DFT）**取点 $\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}$：

$$\hat{a}_j = \sum_{i=0}^{n-1} a_i \, \omega_n^{ij}, \qquad j = 0, 1, \ldots, n-1$$

单位根的关键性质：

- $\omega_n^n = 1$，$\omega_n^{n/2} = -1$；
- **折半引理**：$\omega_{2n}^{2k} = \omega_n^k$，所以长度 $2n$ 的变换可以拆成长度 n 的两个变换；
- **可逆性**：$a_i = \dfrac{1}{n} \displaystyle\sum_{j=0}^{n-1} \hat{a}_j \, \omega_n^{-ij}$。

最后一条说明**逆变换就是把 ω_n 换成 ω_n^{-1}，再整体除以 n**，所以 FFT 只需要写一个函数、加一个方向参数。

3. FFT：分治的实现

把 a 按下标奇偶拆成两半，设

$$E_j = \sum_k a_{2k} \omega_{n/2}^{kj}, \qquad O_j = \sum_k a_{2k+1} \omega_{n/2}^{kj}$$

（分别是偶数项、奇数项的长度 $n/2$ 的 DFT）。由折半引理，

$$\hat{a}_j = E_j + \omega_n^j O_j \qquad (0 \leq j < n/2)$$

又因为 $\omega_n^{j+n/2} = -\omega_n^j$，有

$$\hat{a}_{j+n/2} = E_j - \omega_n^j O_j$$

所以**一次长度 n 的 DFT 由两次长度 $n/2$ 的 DFT 加 $O(n)$ 的合并得到**，递归下去：

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

实现上先用**位反转置换**把序列排成递归所需的顺序，再从长度为 2 开始自底向上合并，就得到前面那 20 行左右的迭代写法。

4. NTT：把单位根换成原根

复数的 ω_n 有精度误差，而且不能直接做模运算。若在模 p 下有元素 g 满足 $g^n = 1$ 且 $g^0, \dots, g^{n-1}$ 两两不同，就能完全照搬上面的推导，只是把 $e^{2\pi i/n}$ 换成 g。

取 g 为模 p 的原根，则 $g^{(p-1)/n}$ 恰好是「n 次单位根」（当 $n \mid p-1$）。于是：

- 要求 $2^k \mid p-1$，k 至少要满足 $n \le 2^k$；
- 常用的 $p = 998244353 = 119 \cdot 2^{23} + 1$，原根 3，所以支持到 2^{23} 的长度；
- 逆变换把 g 换成 g^{-1}（即 g^{p-2}），最后乘 n^{-1}。

除了「用整数模运算代替复数」，其余（位反转、蝴蝶合并、逆变换）和 FFT 一模一样。

5. 任意模数 NTT

目标模数 M 例如 10^9+7，它不满足 $2^k \mid M-1$，没法在模 M 下直接跑 NTT。做法是：

取三个 NTT 友好素数分别算出真实系数的余数，再用中国剩余定理把真实系数还原出来：

$$p_1 = 998244353, \quad p_2 = 1004535809, \quad p_3 = 469762049$$

$$P = p_1 p_2 p_3 \approx 4.7 \times 10^{26}$$

设真实系数为 c。三素数乘积 P 远大于 c 的上界时，c 就是 $c \bmod P$ 的唯一代表元，于是先算出 $r_i = c \bmod p_i$，再做 CRT 得 c，最后对 M 取模即可。

上界：系数 $a_i, b_i \in [0, M)$ 时，卷积系数

$$c \le n\,(M-1)^2$$

取 $M \le 10^9$、$n \le 10^6$ 有 $c < 10^{24} < P$，所以三素数够用（两素数只有约 10^{18}，不够）。

CRT 用两步增量合并即可（设 $x \equiv r_1 \pmod{p_1}$、$x \equiv r_2 \pmod{p_2}$、$x \equiv r_3 \pmod{p_3}$）：

$$t_1 \equiv (r_2 - r_1)\,p_1^{-1} \pmod{p_2}, \qquad x_1 = r_1 + p_1 t_1$$

$$t_2 \equiv (r_3 - x_1)\,(p_1 p_2)^{-1} \pmod{p_3}, \qquad x = x_1 + p_1 p_2 t_2$$

最后只要 $\big(r_1 + p_1 t_1 + p_1 p_2 t_2\big) \bmod M$，不必真的把 x 完整算出来。

6. 复杂度、精度与坑

方案	适用条件 / 注意点
FFT	复数运算，$O(n\log n)$；用 `double`，结果要 `llround` 修正；系数或长度大了有精度风险
NTT	结果本来就取模、且模数满足 $2^k \mid p-1$（如 998244353）；整数运算无误差
任意模数 NTT	模数任意，但要用三个 NTT 素数 + CRT，常数约为普通 NTT 的 3 倍
长度	必须是 2 的幂，取 $\geq \deg a + \deg b + 1$；998244353 下上限 2^{23}
逆变换	别忘了最后整体除以 n（乘 n^{-1}）
精度	FFT 误差随 n 和系数大小增长；保守界限：| 系数	$< 10^5$、$n < 10^6$ 时安全，超了换任意模数 NTT

常见坑：

- 长度开小了会折叠（循环卷积），结果全错；
- 任意模数 NTT 的 CRT 中间量 $p_1p_2 \approx 10^{18}$，用 `long long` 时注意别先乘后取模搞溢出；
- 多项式的「次数 + 1」才是项数，输入输出下标别差一位。

5.13 欧拉筛

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 5;
int p[N], phi[N], cnt;//phi表示欧拉函数
bool vis[N];

void init(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            p[++cnt] = i;
            phi[i] = i - 1;
        }
        for (int j = 1; j <= cnt && i * p[j] <= n; j++) {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) {
                phi[i * p[j]] = phi[i] * p[j];
```

```
                break;
            }
            phi[i * p[j]] = phi[i] * (p[j] - 1);
        }
    }
}
```

5.14 矩阵快速幂

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int MAXN=105,mod=1e9+7;
int n,m,k;
// 普通矩阵乘法: C[i][j] = sum_k A[i][k]*B[k][j] (mod)
// 单位元是主对角线为 1、其余为 0 的矩阵
struct Mat{
    int sz,a[MAXN][MAXN];
    Mat(int s=0):sz(s){up(i,0,sz-1)up(j,0,sz-1)a[i][j]=0;}
    static Mat I(int s){Mat r(s);up(i,0,s-1)r.a[i][i]=1;return r;}
};
Mat operator*(const Mat&A,const Mat&B)
{
    Mat C(A.sz);
    up(i,0,A.sz-1)up(k,0,A.sz-1){
        if(!A.a[i][k])continue;//模意义下为 0 的项直接跳过
        up(j,0,B.sz-1)C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k][j])%mod;
    }
    return C;
}
Mat qpow(Mat A,int b)//A^b, 复杂度 O(sz^3 log b)
{
    Mat res=Mat::I(A.sz);
    while(b){
        if(b&1)res=res*A;
        A=A*A,b>>=1;
    }
    return res;
}
```

```
// 例: 斐波那契 f[1]=f[2]=1, 求 f[b]。
// 转移矩阵 [[1,1],[1,0]] 的 b-1 次幂的左上角就是 f[b]
void solve(){
    int b;cin>>b;
    Mat A(2);
    A.a[0][0]=A.a[0][1]=A.a[1][0]=1,A.a[1][1]=0;
    cout<<qpow(A,b-1).a[0][0]<<'\n';
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--){
        solve();
    }
//    system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
6
1
2
3
10
50
1000000000
// 1
// 1
// 2
// 55
// 586268941
// 21

MAXN 按题目矩阵规模改, 常见不超过 105。
*/
```

5.15 矩阵快速幂（加速 DP 取 max）

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int MAXS=64,NEG=-(1ll<<60);
int n,m,k,R;
int a[8],c[8];
int dmg[1<<6],cost[1<<6];//每种"本轮使用集合"的伤害与基础花费
// 广义矩阵乘法（max-plus）：C[i][j] = max_k (A[i][k] + B[k][j])
// 单位元是主对角线为 0、其余为 -inf 的矩阵；-inf 表示该转移不存在
struct Mat{
    int sz,v[MAXS][MAXS];
    Mat(int s=0):sz(s){up(i,0,sz-1)up(j,0,sz-1)v[i][j]=NEG;}
    static Mat I(int s){Mat r(s);up(i,0,s-1)r.v[i][i]=0;return r;}
};
Mat operator*(const Mat&A,const Mat&B)
{
    Mat C(A.sz);
    up(i,0,A.sz-1)up(t,0,A.sz-1){
        if(A.v[i][t]==NEG)continue;//不存在的转移直接跳过
        up(j,0,B.sz-1){
            if(B.v[t][j]==NEG)continue;
            C.v[i][j]=max(C.v[i][j],A.v[i][t]+B.v[t][j]);
        }
    }
    return C;
}
Mat qpow(Mat A,int b)
{
    Mat res=Mat::I(A.sz);
    while(b){
        if(b&1)res=res*A;
        A=A*A,b>>=1;
    }
    return res;
}
// Blood Memories：n 个角色，每轮 m 点能量，角色 i 技能伤害 a[i]、基础花费 c[i]。
// 上一轮用过技能的角色，本轮花费变成 c[i]+k（连续使用不再叠加）；没用过就恢复成 c[i]。
// 每轮每人最多用一次、总花费不超过 m，求 R 轮最大总伤害。
// 状态取"上一轮用过技能的角色集合"（n<=6，共 2^n 种），转移只与状态有关：
```

```
//      M[P][T] = 本轮用集合 T 能拿到的伤害（当 T 的花费不超过 m 时），否则 -inf
// 于是 R 轮就是一个 max-plus 矩阵幂：答案 = max_T (M^R)[0][T]（初始状态是空集）
void solve(){
    cin>>n>>m>>k>>R;
    up(i,0,n-1)cin>>a[i]>>c[i];
    int S=1<<n;
    up(T,0,S-1){
        int d=0,cc=0;
        up(i,0,n-1)if(T>>i&1)d+=a[i],cc+=c[i];
        dmg[T]=d,cost[T]=cc;
    }
    Mat M(S);
    up(P,0,S-1)up(T,0,S-1){
        int need=cost[T]+k*__builtin_popcountll(T&P);//P 里用过的角色要多花 k
        if(need<=m)M.v[P][T]=dmg[T];
    }
    Mat pw=qpow(M,R);
    int ans=0;
    up(T,0,S-1)ans=max(ans,pw.v[0][T]);
    cout<<ans<<'\n';
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--){
        solve();
    }
//    system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
3
3 7 1 5
59 3
13 2
81 4
```

```
5 14 2 9
66 8
20 2
25 4
39 6
57 7
4 13 7 16
18 2
13 5
33 4
7 1
// 490
// 939
// 741

矩阵规模 2^n <= 64，单组复杂度 O(2^{3n} log R)。
*/
```

5.16 线性基（方案还原与删除）

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e5+5,LOG=64;
int n,m,q,k;
typedef struct liner_basis{
    int basis[LOG]={0};//基向量
    bitset<N> src[LOG];//每个基向量由哪些原始元素异或而成
    bitset<N> dead;//已被删除的元素编号
    int val[N]={0};//编号->值
    int tot=0;//已插入的元素个数（编号1..tot）
    int add(int x,int id)//核心插入：值x、编号id；线性无关返回1
    {
        int path[LOG],pc=0,j=-1,y=x;
        for(int i=LOG-1;i>=0;i--){
            if((y>>i)&1){
                if(!basis[i]){j=i;break;}//找到空位，说明线性无关
                y^=basis[i],path[pc++]=i;
            }
```

```
        }
        if(j<0)return 0;//线性相关：删它不影响张成空间，来源不必记
        bitset<N> s;s.set(id);
        up(t,0,pc-1)s^=src[path[t]];//按消元路径异或出这一位的来源
        basis[j]=y,src[j]=s;
        return 1;
    }
    int insert(int x){tot++;val[tot]=x;add(x,tot);return tot;}//返回新元素的编号
    void erase(int id){if(!dead.test(id))dead.set(id);}//O(1)删除，不重建
    void rebuild_all()//只在必要时才重建：用还活着的元素重算线性基
    {
        memset(basis,0,sizeof(basis));
        up(i,0,LOG-1)src[i].reset();
        up(i,1,tot)if(!dead.test(i))add(val[i],i);
    }
    bool which(int x,bitset<N>&res)//x能否被活着的元素异或出来；能则res给出是哪些
    {
        res.reset();
        int y=x;
        for(int i=LOG-1;i>=0;i--){
            if((y>>i)&1){
                if(!basis[i])return 0;
                y^=basis[i],res^=src[i];
            }
        }
        if((res&dead).any()){//答案里用到了已删元素，这时才不得不重建
            rebuild_all();
            return which(x,res);//重建后来源全是活着的，最多递归一层
        }
        return 1;//没碰到已删元素，答案直接可信，不需要任何重建
    }
    bool can(int x){//x能不能被异或出来
        if(dead.none()){//还没删过东西，消到0就一定能表示
            for(int i=LOG-1;i>=0;i--){
                if((x>>i)&1){
                    if(!basis[i])return 0;
                    x^=basis[i];
                }
            }
            return 1;
        }
```

```
65          bitset<N> s;
66          return which(x,s);
67      }
68      vector<int> list_of(bitset<N>&res)//把来源bitset转成编号列表
69      {
70          vector<int>v;
71          up(i,1,tot)if(res.test(i))v.push_back(i);
72          return v;
73      }
74      int qmax()//最大异或和
75      {
76          int res=0;
77          bitset<N> s;
78          for(int i=LOG-1;i>=0;i--){
79              if((res^basis[i])>res)res^=basis[i],s^=src[i];
80          }
81          if((s&dead).any()){//最大值用到了已删元素，重建后再算
82              rebuild_all();
83              return qmax();
84          }
85          return res;
86      }
87  }lb;
88  lb A;//必须开成全局：src是 bitset<N>[64]，放函数里会爆栈
89
90  void solve(){
91      int q;
92      cin>>n>>q;
93      up(i,1,n){int x;cin>>x;A.insert(x);}
94      while(q--){
95          int op,x;
96          cin>>op>>x;
97          if(op==1)A.insert(x);//插入
98          else if(op==2)A.erase(x);//按编号删除（O(1)，不重建）
99          else if(op==4)cout<<A.qmax()<<'\n';//最大异或和
100         else{//查询x能被哪些元素异或出来
101             bitset<N> s;
102             if(!A.which(x,s))cout<<-1<<'\n';
103             else{
104                 vector<int>v=A.list_of(s);
105                 cout<<v.size();
106                 for(auto id:v)cout<<' '<<id;
107                 cout<<'\n';
```

```
            }
        }
    }
}
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//    cin >> t;
    while (t--){
        solve();
    }
//    system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
4 6
1 2 4 8
3 5
4 0
2 1
3 5
4 0
3 6
// 2 1 3     (5 = val[1]^val[3] = 1^4)
// 15        (1^2^4^8)
// -1        (删掉1号后 {2,4,8} 只能凑偶数, 5 凑不出来)
// 14        (2^4^8)
// 2 2 3     (6 = 2^4)

P3812 【模板】线性基 P4570 [BJWC2011] 元素
*/
```

5.17 线性基（贪心法）

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define int long long
mt19937_64 gen(12345);
const int N=5e5+5,mod=1e9+7,inf=1e9+7;
int n,m,q,k;
```

```
int a[N],b[N],c[N];
typedef struct liner_basis{
    int basis[70]={0},cnt=0,d[70]={0};
    bool has_zero=false;
    bool insert(int x){//是否成功插入可用于查询是否能异或出某值
        for(int i=63;i>=0;i--){
            if((x>>i)&1){
                if(!basis[i]){
                    basis[i]=x;
                    return 1;//成功插入
                }
                x^=basis[i];
            }
        }
        has_zero=1;
        return 0;//插入失败
    }
    int qmax() {
        int res = 0;
        for (int i = 63; i >= 0; i--) {
            if ((res ^ basis[i]) > res) {
                res ^= basis[i];
            }
        }
        return res;
    }
    int qmin() {
        for (int i = 0; i <= 63; i++) {
            if (basis[i]) {
                return basis[i];
            }
        }
        return 0; // 整个线性基为空的情况
    }
    void rebuild() {//使用查询前必须重构
        // 从高位到低位，用较低位的基底消去较高位基底中冗余的 1
        for (int i = 63; i >= 0; i--) {
            for (int j = i - 1; j >= 0; j--) {
                if ((basis[i] >> j) & 1) {
                    basis[i] ^= basis[j];
                }
            }
        }
```

```
51          cnt = 0;
52          for (int i = 0; i <= 63; i++) {
53              if (basis[i]) {
54                  d[cnt++] = basis[i];
55              }
56          }
57      }
58      int lower_query(int k) {//查询第k小的值
59          if (has_zero) {
60              k--;
61          }
62          if (k == 0) return 0;
63          if (k >= (1LL << cnt)) return -1;
64          int res = 0;
65          for (int i = 0; i < cnt; i++) {
66              if ((k >> i) & 1) {
67                  res ^= d[i];
68              }
69          }
70          return res;
71      }
72      int upper_query(int k){
73          int total = (1ll << cnt) - 1 + has_zero;
74          if (k <= 0 || k > total) return -1; // 超出范围返回 -1
75          return lower_query(total - k + 1);  // 转化为求第 (total - k + 1)
                小
76      }
77  }lb;
78  //合并把其中一个线性基里的所有有效基底，依次 insert 到另一个线性基里就可以
        了。
79  void solve(){
80      lb a{};
81      cin>>n;
82      for(int i=1;i<=n;i++){
83          int x;
84          cin>>x;
85          a.insert(x);
86      }
87      cout<<a.qmax()<<'\n';
88  }
89  signed main()
90  {
91      ios::sync_with_stdio(0);
```

```
92      cin.tie(0), cout.tie(0);
93      int t = 1;
94  //    cin >> t;
95      while (t--){
96          solve();
97      }
98  //    system("x-www-browser https://ys.mihoyo.com/");
99      return 0;
100 }
101 /*
102 1
103 5
104 2 3
105 0 0
106 4 5
107 0 0
108 0 0
109
110 */
```

5.18 范德蒙德卷积（推导笔记）

引入

范德蒙德卷积是一种合并组合数的式子，主要应用于组合数学的公式推导。

范德蒙德卷积公式

$$\sum_{i=0}^{k}\binom{n}{i}\binom{m}{k-i}=\binom{n+m}{k}$$

证明 考虑用二项式定理证明：

$$\begin{aligned}\sum_{k=0}^{n+m}\binom{n+m}{k}x^k&=(x+1)^{n+m}\\&=(x+1)^n(x+1)^m\\&=\sum_{r=0}^{n}\binom{n}{r}x^r\sum_{s=0}^{m}\binom{m}{s}x^s\\&=\sum_{k=0}^{n+m}\sum_{r=0}^{k}\binom{n}{r}\binom{m}{k-r}x^k\end{aligned}$$

即有：

$$\binom{n+m}{k}=\sum_{r=0}^{k}\binom{n}{r}\binom{m}{k-r}$$

若考虑其组合意义证明：在一个大小为 $n+m$ 的集合中取出 k 个数，可以等于把大小为 $n+m$ 的集合拆成两个集合，大小分别为 n 与 m，然后从 n 中取出 i 个数，从 m 中取出 $k-i$ 个数的方案数。由于我们有了对于 i 的枚举，于是只需要考虑一种拆法，因为不同的拆法之间是等价的。

边界条件与成立条件

公式对两个上指标的取值没有限制。设 n 为非负整数，上指标 a,b 可以是任意实数（甚至复数），此时二项式系数按广义定义

$$\binom{a}{k}=\frac{a(a-1)(a-2)\cdots(a-k+1)}{k!}\quad(k\geq 0),\qquad\binom{a}{k}=0\quad(k<0)$$

则

$$\sum_{k\in\mathbb{Z}}\binom{a}{k}\binom{b}{n-k}=\binom{a+b}{n}$$

仍然成立。求和范围之所以能写成取遍全体整数，是因为有 $\binom{a}{k}=0\ (k<0)$ 与 $\binom{b}{n-k}=0\ (k>n)$ 这两条约定，所以写成 $\sum_{k=0}^{n}$ 是最安全的写法，既不会漏项也不会多项。

若 a,b 都是非负整数，则非零项恰好落在

$$\max(0,\ n-b)\leq k\leq\min(n,\ a)$$

且当 $n>a+b$ 时两边同为 0。需要注意"$k>a$ 的项为零" 只在 a 为非负整数时才成立：例如 $\binom{-1}{k}=(-1)^k$ 对一切 $k\geq 0$ 都非零，此时把上界写成 a 就会出错。

至于为什么可以从非负整数推广到任意实数：固定 n 后，两边都是关于 a,b 的多项式，而由上面的组合意义证明可知它在所有非负整数点上相等，一个有无穷多个零点的多项式恒为零，故恒等式对任意实数 a,b 成立。

推论

推论 1 及证明

$$\sum_{i=-r}^{s}\binom{n}{r+i}\binom{m}{s-i}=\binom{n+m}{r+s}$$

证明与原公式证明相似。

推论 2 及证明

$$\sum_{i=1}^{n}\binom{n}{i}\binom{n}{i-1}=\binom{2n}{n-1}$$

根据基础的组合数学知识推导，有：

$$\sum_{i=1}^{n}\binom{n}{i}\binom{n}{i-1}=\sum_{i=0}^{n-1}\binom{n}{i+1}\binom{n}{i}=\sum_{i=0}^{n-1}\binom{n}{n-1-i}\binom{n}{i}=\binom{2n}{n-1}$$

推论 3 及证明

$$\sum_{i=0}^{n}\binom{n}{i}^2=\binom{2n}{n}$$

根据基础的组合数学知识推导，有：

$$\sum_{i=0}^{n}\binom{n}{i}^2=\sum_{i=0}^{n}\binom{n}{i}\binom{n}{n-i}=\binom{2n}{n}$$

推论 4 及证明

$$\sum_{i=0}^{m}\binom{n}{i}\binom{m}{i}=\binom{n+m}{m}$$

根据基础的组合数学知识推导，有：

$$\sum_{i=0}^{m}\binom{n}{i}\binom{m}{i}=\sum_{i=0}^{m}\binom{n}{i}\binom{m}{m-i}=\binom{n+m}{m}$$

其中 $\binom{n+m}{m}$ 是我们较为熟悉的网格图路径计数的方案数。所以我们可以考虑其组合意义的证明。

在一张网格图中，从 $(0,0)$ 走到 (n,m) 共走 $n+m$ 步。规定 $(0,0)$ 位于网格图左上角，其中向下走了 n 步，向右走了 m 步，方案数为 $\binom{n+m}{m}$。

换个视角，我们将 $n+m$ 步拆成两部分走，先走 n 步，再走 m 步，那么 n 步中若有 i 步向右，则 m 步中就有 $m-i$ 步向右，故得证。

推论 5 及证明

$$\sum_{i+j+k=n}\binom{a}{i}\binom{b}{j}\binom{c}{k}=\binom{a+b+c}{n}$$

在原公式中把 $n+m$ 看作一个整体，使用两次即可得到。

习题

- CF785D Anton and School - 2
- 洛谷 P2791 幼儿园篮球题

参考资料与注释

1. Vandermonde's Convolution Formula
2. OI Wiki：范德蒙德卷积

5.19 莫比乌斯反演（二维 GCD 求和）

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e6+5,mod=998244353,inf=1e9+7;
int n,m,q,k,cnt;
int p[N],phi[N],pre[N];//phi:欧拉函数 pre:phi的前缀和
bool vis[N];
void init(int n)//线性筛phi
{
    phi[1]=1;
    up(i,2,n){
        if(!vis[i])p[++cnt]=i,phi[i]=i-1;
        for(int j=1;j<=cnt&&i*p[j]<=n;j++){
            vis[i*p[j]]=1;
            if(i%p[j]==0){phi[i*p[j]]=phi[i]*p[j];break;}//p[j]|i, 指数+1
            phi[i*p[j]]=phi[i]*(p[j]-1);
        }
    }
    up(i,1,n)pre[i]=pre[i-1]+phi[i];
}
int gcdsum(int n,int m)//求 sum_{i<=n} sum_{j<=m} gcd(i,j)
{//用 gcd(i,j)=sum_{d|gcd(i,j)} phi(d), 交换求和次序
    if(n>m)swap(n,m);
    int res=0;
    for(int l=1,r;l<=n;l=r+1){//整除分块
```

```
        r=min(n/(n/l),m/(m/l));
        res+=(pre[r]-pre[l-1])*(n/l)*(m/l);
    }
    return res;
}
void solve(){
    cin>>n>>m;
    cout<<gcdsum(n,m)<<'\n';//P2398只有n，读两次n即可
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    init(N-5);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
//  system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
2 2
// 5

P2398 GCD SUM P2568 GCD
*/
```

5.20 莫比乌斯反演（推导笔记）

1. 符号、卷积与莫比乌斯函数

狄利克雷卷积：

$$(f*g)(n)=\sum_{d|n} f(d)\,g\left(\frac{n}{d}\right)$$

满足交换律、结合律，单位元是 $\epsilon(n)=[n=1]$（即 $f*\epsilon=f$）。常用函数：$1(n)=1$（常函数）、$\mathrm{id}(n)=n$、$\epsilon(n)=[n=1]$、μ、φ。

$$\mu(n)=\begin{cases}1, & n=1\\ (-1)^k, & n=p_1p_2\cdots p_k(k\text{ 个互异素数之积})\\ 0, & n\text{ 含平方因子}\end{cases}$$

线性筛递推：$\mu(ip)=0$（$p\mid i$），$\mu(ip)=-\mu(i)$（$p\nmid i$），$\mu(1)=1$；$\varphi(ip)=\varphi(i)\,p$（$p\mid i$），$\varphi(ip)=\varphi(i)(p-1)$（$p\nmid i$）。

---

2. 三条核心恒等式（实战：看到就换，不要现推）

$$\boxed{\mu*1=\epsilon \iff \sum_{d\mid n}\mu(d)=[\,n=1\,]}$$

$$\boxed{\varphi*1=\mathrm{id} \iff \sum_{d\mid n}\varphi(d)=n}$$

$$\boxed{\mu*\mathrm{id}=\varphi \iff \sum_{d\mid n}\mu(d)\,\frac{n}{d}=\varphi(n)}$$

实战意义（这三条就是莫比乌斯反演题的全部工具箱）：

式子里看到	立刻换成	依据	效果
$[\gcd(i,j)=1]$	$\sum_{d\mid\gcd(i,j)}\mu(d)$	$\mu*1=\epsilon$	**消灭互质条件**
$\gcd(i,j)$ 本体	$\sum_{d\mid\gcd(i,j)}\varphi(d)$	$\varphi*1=\mathrm{id}$	**消灭 gcd 本体**
φ 与 μ 混杂	二者互相转化	$\mu*\mathrm{id}=\varphi$	统一成同一种函数

恒等式 1：$n>1$ 时设 n 有 k 个互异素因子，只有无平方因子的 d 有贡献，

$$\sum_{d\mid n}\mu(d)=\sum_{S\subseteq\{1,\dots,k\}}(-1)^{|S|}=(1-1)^k=0$$

恒等式 2（按约分后的分母配对计数）：把 $\dfrac{k}{n}$ $(k=1,\dots,n)$ 共 n 个分数全部约分。约分后分母恰为 b 的分数满足 $b\mid n$ 且分子与 b 互质，有 $\varphi(b)$ 个。按 b 分类即得

$$\sum_{b\mid n}\varphi(b)=n$$

恒等式 3（由前两条推出）：对 $\varphi*1=\mathrm{id}$ 两边同时卷积 μ，用结合律与 $\mu*1=\epsilon$：

$$\mu*(\varphi*1)=\mu*\mathrm{id}\implies\varphi*(\mu*1)=\mu*\mathrm{id}\implies\varphi*\epsilon=\mu*\mathrm{id}\implies\varphi=\mu*\mathrm{id}$$

展开即

$$\varphi(n) = \sum_{d|n} \mu(d)\,\frac{n}{d} = n \sum_{d|n} \frac{\mu(d)}{d}$$

用法：式子里 φ 和 μ 同时出现时，用它把其中一个换掉，统一成同一种函数再合并求和。

---

3. 莫比乌斯反演

$$F = f * 1 \iff f = F * \mu \qquad \text{即} \qquad F(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)\, F\Big(\frac{n}{d}\Big)$$

证明（一行）：μ 就是 1 的狄利克雷逆（$\mu * 1 = \epsilon$），所以

$$F * \mu = (f * 1) * \mu = f * (\mu * 1) = f * \epsilon = f$$

反向同理（两边再卷 1 即可）。

通俗说法就是**" 两边同卷 μ"**，逐项展开即

$$\sum_{d|n} \mu(d)\, F\Big(\frac{n}{d}\Big) = \sum_{d|n} \mu(d) \sum_{e|n/d} f(e) = \sum_{e|n} f(e) \sum_{d|n/e} \mu(d) = \sum_{e|n} f(e)\,[\,n/e = 1\,] = f(n)$$

---

4. 核心套路（三步走）

1. **替换**：用第 2 节的恒等式，把 $[\gcd = 1]$ 或 gcd 本体展开成约数求和；
2. **交换求和次序**，把 d 提到最外层：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d|\gcd(i,j)} h(d) = \sum_{d=1}^{\min(n,m)} h(d)\,\Big\lfloor\frac{n}{d}\Big\rfloor\,\Big\lfloor\frac{m}{d}\Big\rfloor$$

   因为 $[1,n]$ 中 d 的倍数恰有 $\lfloor n/d \rfloor$ 个；
3. **整除分块** + h 的前缀和，单次询问 $O(\sqrt{n})$。

整除分块：$\lfloor n/d \rfloor$ 只有 $O(\sqrt{n})$ 种取值。左端点为 l 时该段右端点

$$r = \min\left(\left\lfloor\frac{n}{\lfloor n/l\rfloor}\right\rfloor,\ \left\lfloor\frac{m}{\lfloor m/l\rfloor}\right\rfloor\right)$$

在 $[l,r]$ 内 $\lfloor n/d\rfloor, \lfloor m/d\rfloor$ 都不变。

---

5. 板子一：二维 GCD 求和

求 $\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j)$。

替换：看到 $\gcd(i,j)$ **本体**，用 $\varphi * 1 = \mathrm{id}$ 无脑替换：

$$\gcd(i,j) = \sum_{d\mid\gcd(i,j)} \varphi(d)$$

交换次序：

$$\boxed{\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j) = \sum_{d=1}^{\min(n,m)} \varphi(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor}$$

预处理 φ 的前缀和，整除分块即可，$O(\sqrt{n})$ 单次询问。

推论（常考，仪仗队 P2158）：$\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=1] = 2\sum_{i=1}^{n}\varphi(i) - 1$。

---

6. 板子二：特定 GCD 计数（ZAP-Queries）

求 $\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=k]$，共 Q 次询问。

先除 k（把 k 从 gcd 里拿出来）：$[\gcd(i,j)=k] = [\gcd(i/k,\, j/k)=1]$，令 $n' = \lfloor n/k\rfloor$，$m' = \lfloor m/k\rfloor$，则原式化为 $\sum_{i\le n'}\sum_{j\le m'}[\gcd(i,j)=1]$。

替换：看到 $[\gcd=1]$，用 $\mu * 1 = \epsilon$ 无脑替换：

$$[\gcd(i,j)=1] = \sum_{d\mid\gcd(i,j)} \mu(d)$$

交换次序：

$$\boxed{\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=k] = \sum_{d=1}^{\min(n',m')} \mu(d)\left\lfloor\frac{n'}{d}\right\rfloor\left\lfloor\frac{m'}{d}\right\rfloor = \sum_{d=1}^{\min(n,m)/k} \mu(d)\left\lfloor\frac{n}{kd}\right\rfloor\left\lfloor\frac{m}{kd}\right\rfloor}$$

预处理 μ 的前缀和，每次询问整除分块 $O(\sqrt{\min(n,m)})$。

---

7. 常考公式速查

用途	公式
卷积定义	$(f*g)(n)=\sum_{d\mid n} f(d)g(n/d)$；单位元 $\epsilon(n)=[n=1]$
$\mu*1=\epsilon$	$\sum_{d\mid n}\mu(d)=[n=1]$（**消灭互质条件**）
$\varphi*1=\mathrm{id}$	$\sum_{d\mid n}\varphi(d)=n$（**消灭 gcd 本体**）
$\mu*\mathrm{id}=\varphi$	$\sum_{d\mid n}\mu(d)\frac{n}{d}=\varphi(n)$（由前两条推出，$\varphi$ 与 μ 互转）
反演定理	$F=f*1 \iff f=F*\mu$，即 $f(n)=\sum_{d\mid n}\mu(d)F(n/d)$
核心套路	$\sum_i\sum_j\sum_{d\mid\gcd(i,j)}h(d)=\sum_d h(d)\lfloor n/d\rfloor\lfloor m/d\rfloor$
GCD 求和	$\sum_i\sum_j\gcd(i,j)=\sum_d\varphi(d)\lfloor n/d\rfloor\lfloor m/d\rfloor$
GCD 计数	$\sum_i\sum_j[\gcd(i,j)=k]=\sum_{d\le\min(n,m)/k}\mu(d)\lfloor n/(kd)\rfloor\lfloor m/(kd)\rfloor$
互质数对	$\sum_{i,j\le n}[\gcd(i,j)=1]=2\sum_{i\le n}\varphi(i)-1$
整除分块	$r=\min\big(\lfloor n/\lfloor n/l\rfloor\rfloor,\lfloor m/\lfloor m/l\rfloor\rfloor\big)$，共 $O(\sqrt{n})$ 段
线性筛递推	$\mu(ip)=0\,(p\mid i)$，$\mu(ip)=-\mu(i)\,(p\nmid i)$；$\varphi(ip)=\varphi(i)p\,(p\mid i)$，$\varphi(ip)=\varphi(i)(p-1)\,(p\nmid i)$
复杂度	筛 $O(N)$；单次询问 $O(\sqrt{n})$（必须配合前缀和）

---

8. 陷阱

- $[n=1]=\sum_{d\mid n}\mu(d)$ 是**求和**，不能把 $[\gcd(i,j)=1]$ 直接写成 $\mu(\gcd(i,j))$；
- 替换后**一定要交换求和次序**，把 d 提到最外层才能用整除分块；顺序反了式子就卡住；
- 整除分块的右端点必须**同时**看 n 和 m 两个下取整，只取一个会算错；
- 多组询问一定要预处理前缀和，否则每次询问退化成 $O(n)$；
- $k>\min(n,m)$ 时答案为 0（此时 $n'=0$，分块循环不执行，自然输出 0）；
- 筛的规模 N 要不小于题目中的 $\max(n,m)$；ZAP 除完 k 后上界更小；
- 答案与 φ/μ 的前缀和都可能很大，全程用 `long long`。

例题 ：P2398 GCD SUM · P2568 GCD · P2158 仪仗队 · P3455 [POI2007] ZAP-Queries · P3327 [SDOI2015] 约数个数和 · P1829 [国家集训队] Crash 的数字表格。

5.21 莫比乌斯反演（特定 GCD 计数）

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=5e4+5,mod=998244353,inf=1e9+7;
int n,m,q,k,cnt;
int p[N],mu[N],pre[N];//mu:莫比乌斯函数 pre:mu的前缀和
bool vis[N];
void init(int n)//线性筛mu
{
    mu[1]=1;
    up(i,2,n){
        if(!vis[i])p[++cnt]=i,mu[i]=-1;
        for(int j=1;j<=cnt&&i*p[j]<=n;j++){
            vis[i*p[j]]=1;
            if(i%p[j]==0){mu[i*p[j]]=0;break;}//含平方因子
            mu[i*p[j]]=-mu[i];
        }
    }
    up(i,1,n)pre[i]=pre[i-1]+mu[i];
}
int query(int n,int m,int k)//求 sum sum [gcd(i,j)==k]
{//把k除掉，再用 [n==1]=sum_{d|n} mu(d) 反演
    n/=k,m/=k;
    if(n>m)swap(n,m);
    int res=0;
    for(int l=1,r;l<=n;l=r+1){//整除分块
        r=min(n/(n/l),m/(m/l));
        res+=(pre[r]-pre[l-1])*(n/l)*(m/l);
    }
    return res;
}
void solve(){
    cin>>n>>m>>k;
    cout<<query(n,m,k)<<'\n';//P3455
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    init(N-5);
    int t = 1;
```

```
    cin >> t;
    while (t--){
        solve();
    }
//  system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
2
2 2 1
4 6 2
// 3
// 5

P3455 [POI2007] ZAP-Queries
*/
```

5.22 行列式计算

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define int long long
mt19937_64 gen(12345);
const int N=5e5+5,mod=1e9+7,inf=1e9+7;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
void solve(){
    int p;
    cin>>n>>m>>p;
    vector<vector<int>>f(n+1);
    vector<int>from,to,ord;//源点汇点
    vector<int>ind(n+1,0);
    for(int i=1;i<=m;++i){
        int u,v;
        cin>>u>>v;
        f[u].push_back(v);
        ind[v]++;
    }
    queue<int>q;
    for(int i=1;i<=n;i++){
        if(!ind[i])q.push(i),from.push_back(i);
        if(f[i].empty())to.push_back(i);
```

```
25      }
26      int k=from.size();
27      vector<vector<int>>a(k+1,vector<int>(k+1,0));//行列式
28      while(!q.empty()){
29          int u=q.front();q.pop();ord.push_back(u);
30          for(auto v:f[u]){
31              if(!--ind[v])q.push(v);
32          }
33      }
34      vector<int>dp(n+1,0);
35      for(int i=1;i<=k;i++){
36          fill(dp.begin(),dp.end(),0);
37          dp[from[i-1]]=1;
38          for(auto u:ord){
39              for(auto v:f[u])(dp[v]+=dp[u])%=p;
40          }
41          for(int j=1;j<=k;j++)a[i][j]=dp[to[j-1]];
42      }
43      auto calc=[&]()->int{//k*k行列式计算
44      int ans=1;
45          for(int i=1;i<=k;i++){
46              for(int j=i+1;j<=k;j++)if(a[j][i]){
47                  swap(a[i],a[j]);//交换两行
48                  ans=(p-ans)%p;//行列式值取负
49                  break;
50              }
51
52              ans=ans*a[i][i]%p;
53              int inv=ksm(a[i][i],p-2,p);
54              for(int j=i+1;j<=k;j++)if(a[j][i]){
55                  int t=p-a[j][i]*inv%p;// 计算消元系数 t = -a[j][i] / a[i][i
                        ]     ,t = - (a[j][i] * inv)
56
57                  for(int r=i;r<=k;++r){
58                      a[j][r]=(a[j][r]+t*a[i][r])%p;// 执行行变换: Row_j =
                            Row_j + t * Row_i
59                  }
60              }
61          }
62          return ans;
63      };
64      cout<<calc()<<'\n';
65  }
```

```
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//    cin >> t;
    while (t--){
        solve();
    }
//    system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
https://codeforces.com/contest/167/problem/E

*/
```

6 数据结构

6.1 01trie

```
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
const int MAXN=5e5+5;                  // 元素个数上限，按题目改
const int LOG=31;                      // 值域位宽：值 < 2^31 用 31；long long 全值域用 63
const int MAXNODE=MAXN*(LOG+1)+5;  // 节点数上界：每个数最多新增 LOG 个节点
// 01-trie：维护一个可重集，支持插入/删除/查询与给定值异或的最大值
// 注意 tr 和 cnt 显式用 int32_t：不然会被 #define int long long 拖成 8 字节一个，内存翻倍
struct orz_trie {
    int32_t tr[MAXNODE][2];// 儿子节点编号
    int32_t cnt[MAXNODE];  // 经过该节点的元素个数（删除就是把它减掉）
    int tot=0;             // 已分配的节点数
    int sz=0;              // 当前集合内的元素个数
    void init(){           // 多组数据时调用
        up(i,0,tot){
            tr[i][0]=tr[i][1]=0;
            cnt[i]=0;
        }
        tot=0,sz=0;
    }
```

```
20      bool count(int x){      // x 是否在集合里
21          int p=0;
22          down(i,LOG-1,0){
23              int bit=(x>>i)&1;
24              if(!tr[p][bit])return 0;
25              p=tr[p][bit];
26          }
27          return cnt[p]>0;
28      }
29      void update(int x, int delta){//delta=1插入, delta=-1删除
30          if(delta<0&&!count(x))return;//删不存在的值会凭空建节点、把计数搞成
                负数, 必须挡掉
31          int p=0;
32          down(i,LOG-1,0){
33              int bit=(x>>i)&1;
34              if(tr[p][bit]==0){
35                  tr[p][bit]=++tot;
36              }
37              p=tr[p][bit];
38              cnt[p]+=delta;
39          }
40          sz+=delta;
41      }
42      int query(int x){       // 集合内元素与 x 异或的最大值
43          if(sz==0)return 0;//空集
44          int p=0, ma=0;
45          down(i,LOG-1,0){
46              int bit=(x>>i)&1;
47              if(tr[p][bit^1]&&cnt[tr[p][bit^1]]>0){//优先走相反位
48                  ma|=1ll<<i;
49                  p=tr[p][bit^1];
50              }else{
51                  p=tr[p][bit];
52              }
53          }
54          return ma;
55      }
56  }tri;
57  // 常见变体: 查询第 k 小、统计异或和小于 x 的个数, 都是在 query 的骨架上按
        cnt 走
```

6.2 dsu

```cpp
#include <vector>
#include <numeric>
#include <algorithm>
using namespace std;
struct DSU {
    vector<int> pa;
    vector<int> sz; // sz[i] 表示以 i 为根的集合大小
    int set_count;       // 当前的集合（连通块）数量
    DSU(int n) {
        pa.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(pa.begin(), pa.end(), 0);
        set_count = n; // 初始时有 n 个独立集合（这里默认忽略索引 0 或者将
            0 当作独立点视具体题意而定，通常可以直接用）
    }
    int find(int x) {
        if (pa[x] == x) {
            return x;
        }
        return pa[x] = find(pa[x]);
    }
    bool join(int x, int y) {
        int root_x = find(x);
        int root_y = find(y);
        if (root_x == root_y) {
            return false;
        }
        // 启发式合并：保证 root_x 是较大的集合
        if (sz[root_x] < sz[root_y]) {
            std::swap(root_x, root_y);
        }
        pa[root_y] = root_x;
        sz[root_x] += sz[root_y];
        set_count--;

        return true;
    }
    // 查询 x 所在集合的大小
    int get_size(int x) {
        return sz[find(x)];
    }
    // 查询当前还有多少个集合
```

```
    int get_set_count() const {
        return set_count;
    }
};
```

6.3 主席树（区间 MEX）

```
#pragma GCC optimize("O3,unroll-loops")
#include <iostream>
using namespace std;
const int N = 2e5 + 5;
int n, q, cnt, root[N];
int ls[N * 40], rs[N * 40], min_pos[N * 40];
int update(int pre, int l, int r, int x, int pos) {
    int u = ++cnt;
    ls[u] = ls[pre]; rs[u] = rs[pre];
    if (l == r) {
        min_pos[u] = pos;
        return u;
    }
    int mid = (l + r) >> 1;
    if (x <= mid) ls[u] = update(ls[pre], l, mid, x, pos);
    else rs[u] = update(rs[pre], mid + 1, r, x, pos);
    min_pos[u] = min(min_pos[ls[u]], min_pos[rs[u]]);
    return u;
}
int query(int u, int l, int r, int limit_L) {
    if (l == r) return l;
    int mid = (l + r) >> 1;
    if (min_pos[ls[u]] < limit_L) return query(ls[u], l, mid, limit_L);
    else return query(rs[u], mid + 1, r, limit_L);
}
void solve() {
    cin >> n >> q;
    for (int i = 1, x; i <= n; ++i) {
        cin >> x;
        if (x > n + 1) {
            root[i] = root[i - 1];
        } else {
            root[i] = update(root[i - 1], 0, n + 1, x, i);
        }
    }
    while (q--) {
```

```
        int l, r;
        cin >> l >> r;
        cout << query(root[r], 0, n + 1, l) << '\n';
    }
}
int main() {
    ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
    return 0;
}
```

6.4 主席树（区间元素数）

```
#include<iostream>
#include<stack>
#include<string>
#include<set>
#include<algorithm>
#include<queue>
#include<map>
#include<vector>
#include<cmath>
using namespace std;
//#define int long long
const int N=1e6+5;
int n,m,cnt,root[N],last[N];
int ls[N<<5],rs[N<<5],w[N<<5];
int update(int lu,int l,int r,int x){
    int u=++cnt;
    ls[u]=ls[lu],rs[u]=rs[lu],w[u]=w[lu];w[u]++;
    if(l==r)return u;
    int mid=(l+r)>>1;
    if(x<=mid)ls[u]=update(ls[lu],l,mid,x);
    else rs[u]=update(rs[lu],mid+1,r,x);
    return u;
}
int query(int u1,int u2,int l,int r,int x){
    if(l>=x)return 0;
    if(r<x)return w[u2]-w[u1];
    int mid=(l+r)>>1;
    return query(ls[u1],ls[u2],l,mid,x)+query(rs[u1],rs[u2],mid+1,r,x);
}
void solve(){
```

```
    cin>>n;
    for(int i=1;i<=n;i++){
        int x;
        cin>>x;
        if(last[x])root[i]=update(root[i-1],0,n,last[x]);
        else root[i]=update(root[i-1],0,n,0);
        last[x]=i;
    }
    cin>>m;
    while(m--){
        int l,r;
        cin>>l>>r;
        cout<<query(root[l-1],root[r],0,n,l)<<'\n';
    }
}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0),cout.tie(0);
    int T=1;
    while(T--)solve();
}
/*
*/
```

6.5 主席树（区间第 K 小）

```
#pragma GCC optimize("O3,unroll-loops")
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
const int N = 2e5 + 5;
int n, m, cnt, root[N], a[N];
int ls[N << 5], rs[N << 5], w[N << 5];
vector<int> b;
int update(int lu, int l, int r, int x) {
    int u = ++cnt;
    ls[u] = ls[lu]; rs[u] = rs[lu]; w[u] = w[lu] + 1;
    if (l == r) return u;
    int mid = (l + r) >> 1;
    if (x <= mid) ls[u] = update(ls[lu], l, mid, x);
    else rs[u] = update(rs[lu], mid + 1, r, x);
```

```
    return u;
}
// 核心变化：前缀和相减，二分找第 K 小
int query(int u1, int u2, int l, int r, int k) {
    if (l == r) return l;
    int d = w[ls[u2]] - w[ls[u1]];
    int mid = (l + r) >> 1;
    if (k <= d) return query(ls[u1], ls[u2], l, mid, k);
    else return query(rs[u1], rs[u2], mid + 1, r, k - d);
}
void solve() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        b.push_back(a[i]);
    }
    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    int max_val = b.size();
    for (int i = 1; i <= n; i++) {
        int x = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
        root[i] = update(root[i - 1], 1, max_val, x);
    }

    while (m--) {
        int l, r, k;
        cin >> l >> r >> k;
        int ans_rk = query(root[l - 1], root[r], 1, max_val, k);
        cout << b[ans_rk - 1] << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

6.6 区间 mex

```
#include<iostream>
#include<stack>
```

```
#include<string>
#include<set>
#include<algorithm>
#include<queue>
#include<map>
#include<vector>
#include<cmath>
#include<cstring>
#include<cstdio>
#include<random>
#include<cmath>
#include <functional>
#include<cctype>
#include<bitset>
#include <ext/pb_ds/assoc_container.hpp> // 包含所有容器（哈希表、树、堆）
#include <ext/pb_ds/tree_policy.hpp>     // 包含树的相关策略（用于实现平衡树）
#include <ext/pb_ds/hash_policy.hpp>     // 包含哈希表的相关策略
#include <ext/pb_ds/priority_queue.hpp>  // 包含优先级队列（堆）
using namespace std;
using namespace __gnu_pbds;
using ll = long long;
template <typename T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=5e5+5,mod=998244353,inf=1e9+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
int gcd(int x,int y)
{
    if(y==0)return x;
    return gcd(y,x%y);
}
int ksm(int a,int n,int p)
{
    if(n<0)return 0;
    int res=1;
    while(n)
    {
        if(n&1)res=res*a%p;
```

```
44          n>>=1;
45          a=a*a%p;
46      }
47      return res;
48  }
49  struct code{
50      int l,r,id;
51      friend bool operator<(code a,code b){
52          return a.r<b.r;
53      }
54  }p[N];
55  struct node{
56      int l,r,w;
57  }t[N<<2];
58  void pushup(int u){
59      t[u].w=min(t[u<<1].w,t[u<<1|1].w);
60  }
61  void build(int u,int l,int r){
62      t[u]={l,r,0};
63      if(l==r)return ;
64      int m=(l+r)>>1;
65      build(u<<1,l,m),build(u<<1|1,m+1,r);
66      pushup(u);
67  }
68  void update(int u,int p,int x){
69      if(t[u].l==t[u].r){t[u].w=x;return ;}
70      int m=(t[u].l+t[u].r)>>1;
71      if(p<=m)update(u<<1,p,x);
72      else update(u<<1|1,p,x);
73      pushup(u);
74  }
75  int query(int u,int x){
76      if(t[u].w>=x)return inf;
77      if(t[u].l==t[u].r)return t[u].l;
78      if(t[u<<1].w<x)return query(u<<1,x);
79      return query(u<<1|1,x);
80  }
81
82  void solve(){
83      cin>>n;
84      up(i,1,n)cin>>a[i];
85      cin>>m;
86      vector<int>ans(m+1,0);
```

```
    up(i,1,m){
        cin>>p[i].l>>p[i].r;
        p[i].id=i;
    }
    sort(p+1,p+m+1);
    int pos=1;
    build(1,1,n);
    up(i,1,m){
        up(j,pos,p[i].r)if(a[j]<=n){
            update(1,a[j],j);
        }
//          cout<<p[i].r-p[i].l+1<<" "<<query(1,p[i].l)<<'\n';
        ans[p[i].id]=p[i].r-p[i].l+1-query(1,p[i].l);
        if(ans[p[i].id]<0)ans[p[i].id]=0;
        pos=p[i].r;
    }
    up(i,1,m)cout<<ans[i]<<'\n';
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
//  system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
1
3 3 2 6
2 1 2


*/
```

6.7 树状数组

```
template <typename T>
struct Fenwick {
```

```
    int n;
    std::vector<T> tree;
    Fenwick(int n_ = 0) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        tree.assign(n + 1, T{});
    }
    void add(int x, T v) {
        for (; x <= n; x += x & -x) {
            tree[x] += v;
        }
    }
    T query(int x) {
        T ans{};
        for (; x > 0; x -= x & -x) {
            ans += tree[x];
        }
        return ans;
    }
    T query(int l, int r) {
        if (l > r) return T{};
        return query(r) - query(l - 1);
    }
};
```

6.8 线段树

```
#include <bits/stdc++.h>
using namespace std;
long long a[100005];
template <typename T>
struct SegTree {
    struct Node {
        T w, tag;
    };
    int n;
    std::vector<Node> t;
    SegTree(int n_ = 0) { init(n_); }
    SegTree(const std::vector<T>& a) { init(a); }
    void init(int n_) {
        n = n_;
```

```
15          t.assign(4 * n + 1, {0, 0});
16      }
17      void init(const std::vector<T>& a) {
18          n = a.size() - 1; // 默认 a 是 1-indexed
19          t.assign(4 * n + 1, {0, 0});
20          auto build = [&](auto &&self, int u, int l, int r) -> void {
21              if (l == r) { t[u].w = a[l]; return; }
22              int mid = (l + r) >> 1;
23              self(self, u << 1, l, mid);
24              self(self, u << 1 | 1, mid + 1, r);
25              pushup(u);
26          };
27          build(build, 1, 1, n);
28      }
29      inline void pushup(int u) {
30          t[u].w = t[u << 1].w + t[u << 1 | 1].w;
31      }
32
33      inline void maketag(int u, int len, T x) {
34          t[u].tag += x;
35          t[u].w += len * x;
36      }
37      inline void pushdown(int u, int l, int r) {
38          if (t[u].tag) {
39              int mid = (l + r) >> 1;
40              maketag(u << 1, mid - l + 1, t[u].tag);
41              maketag(u << 1 | 1, r - mid, t[u].tag);
42              t[u].tag = 0;
43          }
44      }
45      void update(int u, int l, int r, int ql, int qr, T x) {
46          if (ql <= l && r <= qr) {
47              maketag(u, r - l + 1, x);
48              return;
49          }
50          pushdown(u, l, r);
51          int mid = (l + r) >> 1;
52          if (ql <= mid) update(u << 1, l, mid, ql, qr, x);
53          if (qr > mid) update(u << 1 | 1, mid + 1, r, ql, qr, x);
54          pushup(u);
55      }
56
57      T query(int u, int l, int r, int ql, int qr) {
```

```
        if (ql <= l && r <= qr) return t[u].w;
        pushdown(u, l, r);
        int mid = (l + r) >> 1;
        T ans = 0;
        if (ql <= mid) ans += query(u << 1, l, mid, ql, qr);
        if (qr > mid) ans += query(u << 1 | 1, mid + 1, r, ql, qr);
        return ans;
    }
    void update(int ql, int qr, T x) { update(1, 1, n, ql, qr, x); }
    T query(int ql, int qr) { return query(1, 1, n, ql, qr); }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    SegTree<long long> tr(a);
    while (m--) {
        int op, x, y;
        long long k;
        cin >> op;
        if (op == 1) {
            cin >> x >> y >> k;
            tr.update(x, y, k);
        } else {
            cin >> x >> y;
            cout << tr.query(x, y) << '\n';
        }
    }
    return 0;
}
```

6.9 线段树（区间 max）

```
template <typename T>
struct SegTree {
    struct Node {
        T w, tag;
    };
    int n;
    std::vector<Node> t;
```

```
8
9       SegTree(int n_ = 0) { init(n_); }
10      SegTree(const std::vector<T>& a) { init(a); }
11
12      void init(int n_) {
13          n = n_;
14          t.assign(4 * n + 1, {0, 0});
15      }
16
17      void init(const std::vector<T>& a) {
18          n = a.size() - 1; // 默认 a 是 1-indexed
19          t.assign(4 * n + 1, {0, 0});
20          auto build = [&](auto &&self, int u, int l, int r) -> void {
21              if (l == r) { t[u].w = a[l]; return; }
22              int mid = (l + r) >> 1;
23              self(self, u << 1, l, mid);
24              self(self, u << 1 | 1, mid + 1, r);
25              pushup(u);
26          };
27          build(build, 1, 1, n);
28      }
29
30      inline void pushup(int u) {
31          // 合并由加法改为求 max
32          t[u].w = std::max(t[u << 1].w, t[u << 1 | 1].w);
33      }
34
35      // 区间加法对 max 的影响无需长度 len
36      inline void maketag(int u, T x) {
37          t[u].tag += x;
38          t[u].w += x;
39      }
40
41      // pushdown 也不再需要 l 和 r 参数
42      inline void pushdown(int u) {
43          if (t[u].tag) {
44              maketag(u << 1, t[u].tag);
45              maketag(u << 1 | 1, t[u].tag);
46              t[u].tag = 0;
47          }
48      }
49
50      void update(int u, int l, int r, int ql, int qr, T x) {
```

```
        if (ql <= l && r <= qr) {
            maketag(u, x);
            return;
        }
        pushdown(u);
        int mid = (l + r) >> 1;
        if (ql <= mid) update(u << 1, l, mid, ql, qr, x);
        if (qr > mid) update(u << 1 | 1, mid + 1, r, ql, qr, x);
        pushup(u);
    }

    T query(int u, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return t[u].w;
        pushdown(u);
        int mid = (l + r) >> 1;

        // 采用分支返回，避免初始化极小值（如 -INF）带来的数据类型边界麻烦
        if (ql <= mid && qr > mid) {
            return std::max(query(u << 1, l, mid, ql, qr),
                            query(u << 1 | 1, mid + 1, r, ql, qr));
        } else if (ql <= mid) {
            return query(u << 1, l, mid, ql, qr);
        } else {
            return query(u << 1 | 1, mid + 1, r, ql, qr);
        }
    }

    void update(int ql, int qr, T x) { update(1, 1, n, ql, qr, x); }
    T query(int ql, int qr) { return query(1, 1, n, ql, qr); }
};
```

6.10 莫队

```
#include <bits/stdc++.h>
using namespace std;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=5e5+5,mod=998244353,inf=1e18+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
struct node {
```

```
    int l, r, id;
    int blk;
    friend bool operator<(const node& a, const node& b) {
        if (a.blk != b.blk) {
            return a.blk < b.blk;
        }
        return (a.blk & 1) ? (a.r < b.r) : (a.r > b.r);
    }
};
void solve(){
    cin>>n;
    up(i,1,n)cin>>a[i];
    cin>>q;
    vector<node>arr(q+1);
    vector<int>res(q+1);
    vector<int>vis(n+1,0);
    int B = sqrt(n);
    for (int i = 1; i <= q; ++i) {
        cin >> arr[i].l >> arr[i].r;
        arr[i].id = i;
        arr[i].blk = arr[i].l / B;
    }
    sort(arr.begin()+1,arr.end());
    int ans=0;
    auto add=[&](int x)->void{
        vis[a[x]]++;
        if(vis[a[x]]%2==0)ans++;
    };
    auto del=[&](int x)->void{
        vis[a[x]]--;
        if(vis[a[x]]%2==1)ans--;
    };
    int l=1,r=0;
    up(i,1,q){
        while(r<arr[i].r)add(++r);
        while(l>arr[i].l)add(--l);

        while(r>arr[i].r)del(r--);
        while(l<arr[i].l)del(l++);
        res[arr[i].id]=ans;
    }
    up(i,1,q)cout<<res[i]<<'\n';

```

```
}
```

7 树论

7.1 倍增

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005;
const int LOG = 20; // log2(100005) 向上取整，通常 20 足够应付 5e5 的数据
vector<vector<int>> f;
int depth[MAXN];
int fa[MAXN][LOG];
void dfs(int u, int parent) {
    depth[u] = depth[parent] + 1;
    fa[u][0] = parent;

    for (int k = 1; k < LOG; k++) {
        fa[u][k] = fa[fa[u][k - 1]][k - 1];
    }
    for (int v : f[u]) {
        if (v != parent) {
            dfs(v, u);
        }
    }
}
int get_lca(int u, int v) {
    if (depth[u] < depth[v]) {
        swap(u, v);
    }
    for (int k = LOG - 1; k >= 0; k--) {
        if (depth[fa[u][k]] >= depth[v]) {
            u = fa[u][k];
        }
    }
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; k--) {
        if (fa[u][k] != fa[v][k]) {
            u = fa[u][k];
            v = fa[v][k];
        }
    }
```

```
    return fa[u][0];
}
void solve() {
    int n, m, root;
    cin >> n >> m >> root;
    f.assign(n + 1, vector<int>());
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        f[u].push_back(v);
        f[v].push_back(u);
    }
    depth[0] = 0;
    dfs(root, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        cout << get_lca(u, v) << "\n";
    }
}
```

7.2 启发式合并

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=5e5+5,mod=998244353,inf=1e9+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
struct orz_trie {
    int tr[N*32][2];
    int cnt[N*32];
    int tot=0;
    void init(){
        up(i,0,tot){
            tr[i][0]=tr[i][1] = 0;
            cnt[i]=0;
        }
        tot=0;
```

```
    }
    void update(int x, int delta){// delta=1 插入, delta=-1 删除
        int p=0;
        down(i,30,0){
            int bit=(x>>i)&1;
            if(tr[p][bit]==0){
                tr[p][bit]=++tot;
            }
            p=tr[p][bit];
            cnt[p]+=delta;
        }
    }
    int query(int x){
        int p=0;
        int ma=0;
        if(cnt[tr[0][0]] ==0&&cnt[tr[0][1]] == 0) return 0;// 树为空
        down(i,30,0){
            int bit=(x>>i)&1;
            if(tr[p][bit^1]&&cnt[tr[p][bit^1]]>0){
                ma|=(1<<i);
                p=tr[p][bit^1];
            } else {
                p=tr[p][bit];
            }
        }
        return ma;
    }
}tri;
void solve(){
    cin>>n;
    up(i,1,n)cin>>a[i];
    vector<vector<int>>f(n+1);
    up(i,1,n-1){
        int u,v;cin>>u>>v;
        f[u].push_back(v);
        f[v].push_back(u);
    }
    int ans=0;
    vector<int>sz(n+1,0),son(n+1,0),pre(n+1,0);
    auto dfs1=[&](auto &&self,int u,int fa)->void{
        sz[u]++;
        int ma=0;
        pre[u]=pre[fa]^a[u];
```

```
64          for(auto v:f[u])if(v!=fa){
65              self(self,v,u);
66              sz[u]+=sz[v];
67              if(sz[v]>ma){ma=sz[v];son[u]=v;}
68          }
69      };
70      auto dfs2=[&](auto &&self,int u,int fa,int ok,int lca)->void{
71          if(ok==0)ans=max(ans,tri.query(pre[u]^lca));
72          if(ok==1||ok==-1)tri.update(pre[u],ok);
73          for(auto v:f[u])if(v!=fa){
74              self(self,v,u,ok,lca);
75          }
76      };
77      auto dfs3=[&](auto &&self,int u,int fa,bool ok)->void{
78          for(auto v:f[u])if(v!=fa&&v!=son[u]){
79              self(self,v,u,0);// 处理轻儿子
80          }
81          if(son[u])self(self,son[u],u,1);// 处理重儿子
82          for(auto v:f[u])if(v!=fa&&son[u]!=v){
83              dfs2(dfs2,v,u,0,a[u]);// 查询
84              dfs2(dfs2,v,u,1,a[u]);// 插入
85          }
86          tri.update(pre[u],1);// 处理自己
87          ans=max(ans,tri.query(pre[u]^a[u]));
88          if(!ok){
89              for(auto v:f[u])if(v!=fa)
90                  dfs2(dfs2,v,u,-1,0);// 清空轻儿子
91              tri.update(pre[u],-1);
92          }
93      };
94      dfs1(dfs1,1,0);
95      dfs3(dfs3,1,0,1);
96      cout<<ans<<'\n';
97  }
98
99  signed main()
100 {
101     ios::sync_with_stdio(0);
102     cin.tie(0), cout.tie(0);
103     int t = 1;
104 //    cin >> t;
105     while (t--){
106         solve();
```

```
    }
//    system("x-www-browser https://ys.mihoyo.com/");
//    cout << fixed << setprecision(10) << x << '\n';
    return 0;
}
/*
启发式合并步骤:
预处理子树大小, 确定重儿子;
先递归处理轻儿子, 再处理重儿子;
暴力合并轻儿子信息并统计当前点答案;
当前点为轻儿子时清空贡献。
总复杂度 O(n log n)。
*/
```

8 计算几何

8.1 凸包（单调链法）

```
#pragma GCC optimize("O2")
#pragma GCC optimize("O3")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define sm int,vector<int>,greater<int>
#define int long long
mt19937_64 gen(12345);
const int N=1e6+5,mod=998244353,inf=1e9+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
int gcd(int x,int y)
{
    if(y==0)return x;
    return gcd(y,x%y);
}
int ksm(int a,int n,int p)
{
    if(n<0)return 0;
    int res=1;
```

```
25      while(n)
26      {
27          if(n&1)res=res*a%p;
28          n>>=1;
29          a=a*a%p;
30      }
31      return res;
32  }
33  const double eps = 1e-9;
34  int sgn(double x) {
35      if (abs(x) < eps) return 0;
36      return x < 0 ? -1 : 1;
37  }
38  struct Point {
39      double x, y;
40      Point(double _x = 0, double _y = 0) : x(_x), y(_y) {}
41
42      // 向量减法，用于求叉积
43      Point operator - (const Point& b) const {
44          return Point(x - b.x, y - b.y);
45      }
46
47      //优先按 x 排序，x 相同按 y 排序
48      bool operator < (const Point& b) const {
49          if (sgn(x - b.x) == 0) return sgn(y - b.y) < 0;
50          return sgn(x - b.x) < 0;
51      }
52
53      // 判等（用于去重）
54      bool operator == (const Point& b) const {
55          return sgn(x - b.x) == 0 && sgn(y - b.y) == 0;
56      }
57  };
58
59  // 计算两个向量 a 和 b 的叉积
60  double cross(const Point& a, const Point& b) {
61      return a.x * b.y - a.y * b.x;
62  }
63
64  // 计算向量 p0->p1 和 p0->p2 的叉积
65  // 返回值 > 0：p2 在 p0->p1 的左侧（向左转）
66  // 返回值 < 0：p2 在 p0->p1 的右侧（向右转）
67  // 返回值 == 0：三点共线
```

```cpp
double cross(const Point& p0, const Point& p1, const Point& p2) {
    return cross(p1 - p0, p2 - p0);
}

// 输入点集 P，返回凸包上的点集（逆时针顺序，起点和终点不重合）
vector<Point> convex_hull(vector<Point>& P) {
    int n = P.size();
    if (n <= 2) return P; // 退化情况：点数小于等于2，本身就是凸包

    // 1. 排序并去重
    sort(P.begin(), P.end());
    P.erase(unique(P.begin(), P.end()), P.end());
    n = P.size();
    if (n <= 2) return P;

    vector<Point> hull(2 * n); // 用数组模拟栈，速度最快
    int top = 0;               // 栈顶指针，表示栈内元素个数

    // 2. 求下凸壳
    for (int i = 0; i < n; i++) {
        // 【核心判断】：如果向右转或共线，则弹栈
        // 注意：如果你需要保留凸包边上的共线点，把 <= 改成 < 即可
        while (top >= 2 && sgn(cross(hull[top - 2], hull[top - 1], P[i]))
            <= 0) {
            top--;
        }
        hull[top++] = P[i];
    }

    // 3. 求上凸壳
    int tmp = top; // 记录下凸壳的元素个数，上凸壳弹栈时不能破坏下凸壳
    for (int i = n - 2; i >= 0; i--) {
        while (top > tmp && sgn(cross(hull[top - 2], hull[top - 1], P[i]))
            <= 0) {
            top--;
        }
        hull[top++] = P[i];
    }

    // 4. 收尾：最后加入的起点与栈底的起点重合，需减去
    if (n > 1) top--;
    hull.resize(top); // 截断有效部分
    return hull;
```

```
}

void solve(){
    cin>>n;
    vector<Point>pts(n);
    up(i,0,n-1)
        cin>>pts[i].x>>pts[i].y;
    vector<Point>hull=convex_hull(pts);
    double ans=0;
    double tx=hull.back().x,ty=hull.back().y;
    for(auto const &[x,y]:hull){
        ans+=sqrt((x-tx)*(x-tx)+(y-ty)*(y-ty));
        tx=x;
        ty=y;
    }
    cout << fixed << setprecision(2) << ans << '\n';
}
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
//  system("x-www-browser https://ys.mihoyo.com/");
//  cout << fixed << setprecision(10) << x << '\n';
    return 0;
}
```

8.2 极角排序

```
#include <bits/stdc++.h>
using namespace std;
struct Point {
    long long x, y;

    // 1. 象限划分 / 半平面划分
    // 规定极角从 x 轴正半轴开始，逆时针方向为正。
    // 我们将平面分为上下两个半区：
    // 上半区（包含正 x 轴）：y > 0，或者 y == 0 且 x > 0（返回 0）
    // 下半区（包含负 x 轴）：y < 0，或者 y == 0 且 x < 0（返回 1）
```

```
    // 原点 (0,0) 没有方向，按下面的判断会落到下半区 (half=1)；题目含原点时建议单独处理
    int half() const {
        if (y > 0 || (y == 0 && x > 0)) return 0;
        return 1;
    }
};

// 2. 叉积 (Cross Product): OA × OB
// 几何意义：以 OA 和 OB 为邻边的平行四边形的有向面积
// 如果结果 > 0，说明从 OA 到 OB 是逆时针方向 (A 的极角小于 B 的极角)
// 如果结果 < 0，说明是顺时针方向
// 如果结果 == 0，说明 A, O, B 三点共线
long long cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

// 3. 距离平方 (用于共线时的副关键字排序)
long long dist2(Point a) {
    return a.x * a.x + a.y * a.y;
}

// 4. 极角排序比较函数 (逆时针)
bool polar_cmp(Point a, Point b) {
    // 步骤一：如果两个点在不同的半区，上半区的点一定排在前面
    if (a.half() != b.half()) {
        return a.half() < b.half();
    }

    // 步骤二：如果在同一个半区，用叉积判断相对位置
    long long cp = cross(a, b);
    if (cp != 0) {
        // cp > 0 表示 a 在 b 的顺时针方向 (即 a 的角度更小)
        return cp > 0;
    }

    // 步骤三：如果极角完全相同 (共线)，通常把距离原点更近的点排在前面
    return dist2(a) < dist2(b);
}

// ==========================================
// 5. 用法示例
// ==========================================
```

```
void polar_sort_example() {
    vector<Point> pts = {
{-2, -2}, {1, 1}, {-1, 1}, {2, 0}, {0, 2}, {1, -1}
};

    // 假设以原点 (0,0) 为基准点进行极角排序
    // 如果基准点是其他的点 P，只需要把所有的点先减去 P（平移坐标系）再排序即可
    sort(pts.begin(), pts.end(), polar_cmp);

    // 排序后的 pts 顺序将是严格的逆时针一圈
}
```

8.3 求凸包直径（旋转卡壳）

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp> // 包含所有容器（哈希表、树、堆）
#include <ext/pb_ds/tree_policy.hpp>     // 包含树的相关策略（用于实现平衡树）
#include <ext/pb_ds/hash_policy.hpp>     // 包含哈希表的相关策略
//#include <ext/pb_ds/priority_queue.hpp>  // 包含优先级队列（堆）
using namespace std;
using namespace __gnu_pbds;
using ll = long long;
template <typename T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define sm int,vector<int>,greater<int>
#define int long long
mt19937_64 gen(12345);
const int N=1e6+5,mod=998244353,inf=1e9+7;;
int n,m,q,k;
int a[N],b[N],c[N],d[N];
int gcd(int x,int y)
{
    if(y==0)return x;
    return gcd(y,x%y);
}
int ksm(int a,int n,int p)
{
    if(n<0)return 0;
    int res=1;
```

```
    while(n)
    {
        if(n&1)res=res*a%p;
        n>>=1;
        a=a*a%p;
    }
    return res;
}
const double eps = 1e-9;
int sgn(double x) {
    if (abs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
struct Point {
    double x, y;
    Point(double _x = 0, double _y = 0) : x(_x), y(_y) {}

    // 向量减法，用于求叉积
    Point operator - (const Point& b) const {
        return Point(x - b.x, y - b.y);
    }

    //优先按 x 排序，x 相同按 y 排序
    bool operator < (const Point& b) const {
        if (sgn(x - b.x) == 0) return sgn(y - b.y) < 0;
        return sgn(x - b.x) < 0;
    }

    // 判等（用于去重）
    bool operator == (const Point& b) const {
        return sgn(x - b.x) == 0 && sgn(y - b.y) == 0;
    }
};

// 计算两个向量 a 和 b 的叉积
double cross(const Point& a, const Point& b) {
    return a.x * b.y - a.y * b.x;
}

// 计算向量 p0->p1 和 p0->p2 的叉积
// 返回值 > 0: p2 在 p0->p1 的左侧（向左转）
// 返回值 < 0: p2 在 p0->p1 的右侧（向右转）
// 返回值 == 0: 三点共线
```

```cpp
double cross(const Point& p0, const Point& p1, const Point& p2) {
    return cross(p1 - p0, p2 - p0);
}

int dist(const Point& p0, const Point& p1){
    return (p0.x-p1.x)*(p0.x-p1.x)+(p0.y-p1.y)*(p0.y-p1.y);
}
// 输入点集 P，返回凸包上的点集（逆时针顺序，起点和终点不重合）
vector<Point> convex_hull(vector<Point>& P) {
    int n = P.size();
    if (n <= 2) return P; // 退化情况：点数小于等于2，本身就是凸包

    // 1. 排序并去重
    sort(P.begin(), P.end());
    P.erase(unique(P.begin(), P.end()), P.end());
    n = P.size();
    if (n <= 2) return P;

    vector<Point> hull(2 * n); // 用数组模拟栈，速度最快
    int top = 0;               // 栈顶指针，表示栈内元素个数

    // 2. 求下凸壳
    for (int i = 0; i < n; i++) {
        // 【核心判断】：如果向右转或共线，则弹栈
        // 注意：如果你需要保留凸包边上的共线点，把 <= 改成 < 即可
        while (top >= 2 && sgn(cross(hull[top - 2], hull[top - 1], P[i]))
            <= 0) {
            top--;
        }
        hull[top++] = P[i];
    }

    // 3. 求上凸壳
    int tmp = top; // 记录下凸壳的元素个数，上凸壳弹栈时不能破坏下凸壳
    for (int i = n - 2; i >= 0; i--) {
        while (top > tmp && sgn(cross(hull[top - 2], hull[top - 1], P[i]))
            <= 0) {
            top--;
        }
        hull[top++] = P[i];
    }

    if (n > 1) top--;
```

```
111     hull.resize(top);
112     return hull;
113 }
114 int hull_diam(vector<Point>& hull){
115     int j=2;
116     int ans=0,H=hull.size();
117     if (H == 2) {
118         return dist(hull[0], hull[1]) ;
119     }
120     if (H <= 1) {
121         return 0;
122     }
123     up(i,0,H-1){
124         int next_i=(i+1)%H;
125         while(cross(hull[i],hull[next_i],hull[j])<=cross(hull[i],hull[
                next_i],hull[(j+1)%H])){
126             j=(j+1)%H;
127         }
128         ans=max(ans,dist(hull[i],hull[j]));
129         ans=max(ans,dist(hull[next_i],hull[j]));
130     }
131     return ans;
132 }
133 void solve(){
134     cin>>n;
135     vector<Point>pts(n);
136     up(i,0,n-1)cin>>pts[i].x>>pts[i].y;
137     vector<Point>hull=convex_hull(pts);
138     cout<<hull_diam(hull);
139 }
140 
141 signed main()
142 {
143     ios::sync_with_stdio(0);
144     cin.tie(0), cout.tie(0);
145     int t = 1;
146     // cin >> t;
147     while (t--){
148         solve();
149     }
150 //  system("x-www-browser https://ys.mihoyo.com/");
151 //  cout << fixed << setprecision(10) << x << '\n';
152     return 0;
```

```
}
/*
1
38133 49107

2 27269


*/
```

8.4 求凸包面积

```cpp
// 传入多边形的顶点数组 p（不要求首尾点重合，只要是顺时针或逆时针按顺序排列即可）
// 返回多边形的面积
double polygon_area(const vector<Point>& p) {
    int n = p.size();
    if (n < 3) return 0.0;
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += cross(p[i], p[(i + 1) % n]);
    }
    return abs(sum) / 2.0;
}
```

8.5 闵可夫斯基和与皮克定理

1. 核心模型：平移相交转化

- **问题模型**：求有多少个整数向量 (u, v)，使得放大了 k 倍的凸多边形 kP 在平移 (u, v) 后，与凸多边形 Q 至少有一个公共点。
- **等价结论**：平移相交等价于**闵可夫斯基差**。

$$(u, v) \in Q \oplus (-kP)$$

题目转化为：**求凸多边形 $R_k = Q \oplus (-kP)$ 内部及边界上的“总整点数”。**

---

2. 核心公式推导路线

2.1 皮克定理 (Pick's Theorem) - 离散与连续的桥梁 设 S 为多边形面积，I 为内部整点数，B 为边界整点数。

- **原始公式**：$S = I + \frac{B}{2} - 1$
- **变形式（求总整点数）**：设 $Ans = I + B$

$$Ans = S + \frac{B}{2} + 1 = \frac{2S + B + 2}{2}$$

注：算法中全程维护双倍面积 $2S$ 以避免浮点数精度误差。

2.2 边界整点数 (Boundary Points) 的线性拆分

- **单边整点数**：对于边向量 (dx, dy)，其覆盖的整点数为 $\gcd(|dx|, |dy|)$。
- **闵可夫斯基和性质**：两个多边形相加，边界整点数直接相加。放大 k 倍，边界整点数也放大 k 倍。
- **最终公式**：

$$B(R_k) = B(Q) + k \cdot B(P)$$

2.3 双倍面积 (Double Area) 的一元二次多项式展开

- **混合体积理论**：两个凸多边形 A 和 B 的闵可夫斯基和面积公式为 $2S(A \oplus B) = 2S(A) + 2S(B) + 2M(A, B)$，其中 M 为混合面积常数。
- **放大 k 倍后的面积展开**：$-P$ 放大 k 倍，自身面积乘 k^2；与 Q 的混合面积乘 k。
- **最终公式**：

$$2S(R_k) = 2S(Q) + k^2 \cdot 2S(P) + k \cdot Mix$$

2.4 逆向推导常数 Mix

- 不需要正面计算混合面积，直接代入 $k = 1$ 时的状态反推。
- **反推公式**：

$$Mix = 2S(R_1) - 2S(Q) - 2S(P)$$

注：其中 $R_1 = Q \oplus (-P)$，也就是 $k = 1$ 时的闵可夫斯基和多边形。

---

3. 算法流程

1. **预处理极角与多边形**：
   - 求出点集 A 和 B 的凸包，记为 P 和 Q。
   - 将 P 取反得到 $-P$，并确保其顶点依旧是**逆时针**顺序。
2. **计算基础属性**：

- 算出 $2S(P)$、$2S(Q)$（利用叉积）。
- 算出 $B(P)$、$B(Q)$（利用边长坐标差的 gcd）。

3. **计算 $k=1$ 时的状态**：
   - 利用**双指针**和叉积比较，将 Q 和 $-P$ 的边向量合并，得到多边形 $R_1 = Q \oplus (-P)$ 的所有边。
   - 利用这些边算出 $2S(R_1)$。
4. **常数析出**：

$$Mix = (2S(R_1) - 2S(Q) - 2S(P) + 2 \times MOD) \pmod{MOD}$$

5. $O(1)$ **极速回答询问**：
   对于任意给定的 k，套入多项式（注意取模操作）：
   - $Cur_2S = (2S(Q) + (k^2 \bmod MOD) \cdot 2S(P) + (k \bmod MOD) \cdot Mix) \pmod{MOD}$
   - $Cur_B = (B(Q) + (k \bmod MOD) \cdot B(P)) \pmod{MOD}$
   - **答案**：$Ans = (Cur_2S + Cur_B + 2) \times 499122177 \pmod{MOD}$

   注：499122177 是 2 在模 998244353 下的乘法逆元。

9 杂项

9.1 CDQ 分治

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e5+5,M=2e5+5;
int n,k,cnt;
struct node{int a,b,c,w,res;}p[N],t[N];//w:相同元素个数 res:被多少个点支配
int bit[M],ans[N];//ans[i]:有多少个点被恰好i个点支配
bool cmp1(node x,node y)//按a,b,c排序
{
    if(x.a!=y.a)return x.a<y.a;
    if(x.b!=y.b)return x.b<y.b;
    return x.c<y.c;
}
```

```
bool cmp2(node x,node y)//按b,c排序
{
    if(x.b!=y.b)return x.b<y.b;
    return x.c<y.c;
}
void add(int x,int v){for(;x<=k;x+=x&-x)bit[x]+=v;}
int query(int x){int res=0;for(;x;x-=x&-x)res+=bit[x];return res;}
void cdq(int l,int r)
{
    if(l==r)return;
    int mid=(l+r)>>1;
    cdq(l,mid),cdq(mid+1,r);
    sort(t+l,t+mid+1,cmp2),sort(t+mid+1,t+r+1,cmp2);
    int i=l,j=mid+1;
    for(;j<=r;j++){//双指针把左半区b<=b_j的点插入树状数组
        while(i<=mid&&t[i].b<=t[j].b)add(t[i].c,t[i].w),i++;
        t[j].res+=query(t[j].c);
    }
    up(u,l,i-1)add(t[u].c,-t[u].w);//撤销树状数组
}
void solve(){
    cin>>n>>k;
    up(i,1,n)cin>>p[i].a>>p[i].b>>p[i].c;
    sort(p+1,p+n+1,cmp1);
    up(i,1,n){//去重，相同三元组合并，w记个数
        if(i>1&&p[i].a==p[i-1].a&&p[i].b==p[i-1].b&&p[i].c==p[i-1].c)t[cnt
            ].w++;
        else t[++cnt]=p[i],t[cnt].w=1;
    }
    cdq(1,cnt);
    up(i,1,cnt)ans[t[i].res+t[i].w-1]+=t[i].w;
    up(i,0,n-1)cout<<ans[i]<<'\n';
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
//  cin >> t;
    while (t--){
        solve();
    }
```

```
//  system("x-www-browser https://ys.mihoyo.com/");
    return 0;
}
/*
10 3
3 3 3
2 3 3
2 3 1
3 1 1
3 1 2
1 3 1
1 1 2
1 2 2
1 3 2
1 2 1
// 3 1 3 0 1 0 1 0 0 1（每行一个）

P3810 【模板】三维偏序（陌上花开） P3157 [CQOI2011] 动态逆序对 P2487 [SDOI2011] 拦截导弹
*/
```

9.2 CDQ 分治优化 DP

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define up(i, a, b) for (int i = (a); i <= (b); ++i)
#define down(i, a, b) for (int i = (a); i >= (b); --i)
#define int long long
mt19937_64 gen(12345);
const int N=1e6+5,INF=1e18;
int n,m,timer;
int A[N],C[N];//A[i]=前i个里(红-黄)  C[i]=前i个里(红-蓝)
int dp[N];//dp[i]=前i个位置能得到的最大红色长度
int L[N],R[N];//左半、右半的下标，按A排序后使用
bool cmpA(int x,int y){return A[x]<A[y];}
//带时间戳的前缀最大值树状数组: max没法删除，用timer代替清空
int bit[2*N],vis[2*N];
void upd(int x,int v)
{
    for(;x<=m;x+=x&-x){
        if(vis[x]!=timer)vis[x]=timer,bit[x]=v;//本轮第一次碰到这个位置
        else bit[x]=max(bit[x],v);
```

```
    }
}
int qry(int x)
{
    int res=-INF;
    for(;x;x-=x&-x)if(vis[x]==timer)res=max(res,bit[x]);
    return res;
}
/*
 * 区间 (k,i] 是红色  <=>  段内 红>=黄 且 红>=蓝  <=>  A[i]>=A[k] 且 C[i]>=C[k]
 *
 * dp[i] = max( dp[i-1] ,                                          // 最后一段不是红色，白扔一格
 *              i + max{dp[k]-k : k<i 且 A[k]<=A[i] 且 C[k]<=C[i]} ) // 最后一段 (k,i] 是红色
 *
 * 转移形状是"左边的 k 更新右边的 i"，于是用 CDQ 分治：先算左半，再用左半更新右半，最后算右半内部。
 * 更新时 k<i 已自动满足，只剩 A、C 两维偏序 —— 两半按 A 排序后双指针，
 * 用树状数组按 C 查"前缀最大 dp[k]-k"。总复杂度 O(n log^2 n)。
 */
void cdq(int l,int r)
{
    if(l==r){//递归到单点：所有用到它的转移都已经做完了，只差"最后一段不是红色"这一条
        if(l)dp[l]=max(dp[l],dp[l-1]);
        return;
    }
    int mid=(l+r)>>1;
    cdq(l,mid);//第一步：先把左半的 dp 全部算出来（后面要拿它们当转移来源）

    //第二步：用左半的 k 属于 [l,mid] 去更新右半的 i 属于 [mid+1,r]
    int p=0,q=0;
    up(i,l,mid)L[p++]=i;
    up(i,mid+1,r)R[q++]=i;
    sort(L,L+p,cmpA);//两半都按 A 从小到大排，方便双指针
    sort(R,R+q,cmpA);

    timer++;//开一轮新的树状数组（时间戳+1，等价于清空）
    int ptr=0;
    up(j,0,q-1){
        int i=R[j];
```

```
59          //把所有 A[k]<=A[i] 的 k 插进树状数组: 下标是 C[k], 存 dp[k]-k
60          while(ptr<p&&A[L[ptr]]<=A[i]){
61              int k=L[ptr++];
62              upd(C[k]+n+1,dp[k]-k);
63          }
64          //此时树状数组里全是 A[k]<=A[i] 的 k, 查 C[k]<=C[i] 的最大 dp[k]-k
                即为最优来源
65          int best=qry(C[i]+n+1);
66          if(best>-INF)dp[i]=max(dp[i],i+best);
67      }
68
69      cdq(mid+1,r);//第三步: 再算右半内部的转移
70  }
71  void solve(){
72      cin>>n;
73      up(i,1,n){
74          int r,y,b;cin>>r>>y>>b;
75          A[i]=A[i-1]+r-y;
76          C[i]=C[i-1]+r-b;
77      }
78      m=2*n+2;//树状数组下标 C[i]+n+1 落在 [1,2n+1]
79      up(i,0,n)dp[i]=-INF;
80      dp[0]=0;
81      cdq(0,n);
82      cout<<dp[n]<<'\n';
83  }
84  signed main()
85  {
86      ios::sync_with_stdio(0);
87      cin.tie(0), cout.tie(0);
88      int t = 1;
89  //    cin >> t;
90      while (t--){
91          solve();
92      }
93  ////    system("x-www-browser https://ys.mihoyo.com/");
94      return 0;
95  }
96  /*
97  10
98  1 0 0
99  0 1 1
100 0 1 1
```

```
1 1 0
0 1 1
1 0 1
0 1 0
0 1 1
1 1 1
1 0 1
// 7

n=1e6 实测约 0.7s，内存约 35MB（要开 -O2）
*/
```

9.3 CDQ 分治（推导笔记）

简介

CDQ 分治是一种思想而不是具体的算法，与动态规划类似。目前这个思想的拓展十分广泛，依原理与写法的不同，大致分为三类：

- 解决和点对有关的问题。
- 1D 动态规划的优化与转移。
- 通过 CDQ 分治，将一些动态问题转化为静态问题。

CDQ 分治的思想最早由 IOI2008 金牌得主陈丹琦在高中时整理并总结，它也因此得名。

解决和点对有关的问题

这类问题多数类似于「给定一个长度为 n 的序列，统计有一些特性的点对 (i,j) 的数量」或「给定一个长度为 n 的序列，找到一对点 (i,j) 使得一些函数的值最大」。

CDQ 分治解决这类问题的算法流程如下：

1. 找到这个序列的中点 mid；
2. 将所有点对 (i,j) 划分为 3 类：
   (a) $1 \le i \le mid,\ 1 \le j \le mid$ 的点对；
   (b) $1 \le i \le mid,\ mid+1 \le j \le n$ 的点对；
   (c) $mid+1 \le i \le n,\ mid+1 \le j \le n$ 的点对。
3. 将 $(1,n)$ 这个序列拆成两个序列 $(1,mid)$ 和 $(mid+1,n)$。此时第一类点对和第三类点对都在这两个序列之中；

4. 递归地处理这两类点对；

5. 设法处理第二类点对。

可以看到 CDQ 分治的思想就是不断地把点对通过递归的方式分给左右两个区间。

在实际应用时，我们通常使用一个函数 `solve(l,r)` 处理 $l \le i \le r,\ l \le j \le r$ 的点对。上述算法流程中的递归部分便是通过 `solve(l,mid)` 与 `solve(mid+1,r)` 来实现的。剩下的第二类点对则需要额外设计算法解决。

例题：三维偏序 给定一个序列，每个点有 a_i, b_i, c_i 三个属性，试求：这个序列里有多少对点对 (i, j) 满足 $a_j \le a_i$ 且 $b_j \le b_i$ 且 $c_j \le c_i$ 且 $j \ne i$。

解题思路：三维偏序是 CDQ 分治的经典问题。题目要求统计序列里点对的个数，那试一下用 CDQ 分治。

首先将序列按 a 排序。

假设已经写好了 `solve(l,r)`，并且递归搞定了 `solve(l,mid)` 和 `solve(mid+1,r)`。现在要做的，就是统计满足 $l \le i \le mid$，$mid+1 \le j \le r$ 的点对 (i, j) 中，有多少点对还满足 $a_i \le a_j$，$b_i \le b_j$，$c_i \le c_j$ 的限制条件。

稍微思考一下就会发现，那个 $a_i \le a_j$ 的限制条件没啥用了：既然 i 比 mid 小，j 比 mid 大，那 i 肯定比 j 要小；已经将序列按 a 排序，就一定有 $a_i \le a_j$。现在还剩下两个限制条件：$b_i \le b_j$ 与 $c_i \le c_j$。根据这个限制条件我们就可以枚举 j，求出有多少个满足条件的 i。

为了方便枚举，我们把 (l, mid) 和 $(mid+1, r)$ 中的点全部按照 b 的值从小到大排个序。之后我们依次枚举每一个 j，把所有 $b_i \le b_j$ 的点 i 全部插入到某种数据结构里（这里我们选择树状数组）。此时只要查询树状数组里有多少个点的 c 值是小于等于 c_j 的，我们就求出了对于这个点 j，有多少个 i 可以合法匹配它了。

当我们插入一个 c 值等于 x 的点时，我们就令树状数组的 x 这个位置单点加一，而查询树状数组里有多少个点小于 x 的操作实际上就是在求前缀和。

对于每一个 j，我们都需要将所有 $b_i \le b_j$ 的点 i 插入树状数组中。由于所有的 i 和 j 都已事先按照 b 值排好序，这样的话只要以双指针的方式在树状数组里插入点，则对树状数组的插入操作就能从 $O(n^2)$ 次降到 $O(n)$ 次。

通过这样一个算法流程，我们就用 $O(n \log n)$ 的时间处理完了关于第二类点对的信息了。此时算法的时间复杂度是

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + O(n \log n) = O(n \log^2 n)$$

实现细节：相等元素要先合并。若存在三元组完全相同的点，它们之间互相满足偏序关系，但在分治过程中只有「左半区贡献给右半区」这一个方向，相等的点对会漏统计。因此要先把相同的三元组合并成一个点，用 w 记下它的个数，某一点被支配的总数即为 `res + w - 1`，再把这个数累加到答案数组里（`ans[d]` 表示被恰好 d 个点支配的点的个数）。

CDQ 分治优化 1D/1D 动态规划的转移

1D/1D 动态规划指的是一类特定的 DP 问题，该类题目的特征是 DP 数组是一维的，转移是 $O(n)$ 的。如果条件良好的话，有时可以通过 CDQ 分治来把它们的时间复杂度由 $O(n^2)$ 降至 $O(n\log^2 n)$。

例如，给定一个序列，每个元素有两个属性 a，b。我们希望计算一个 DP 式子的值，它的转移方程如下：

$$dp_i = 1 + \max_{j=1}^{i-1} dp_j \,[a_j < a_i][b_j < b_i]$$

这是一个二维最长上升子序列的 DP 方程，即只有 $j < i$，$a_j < a_i$，$b_j < b_i$ 的点 j 可以更新点 i 的 DP 值。直接转移显然是 $O(n^2)$ 的。

我们发现 dp_j 转移到 dp_i 这种转移关系也是一种点对间的关系，所以我们用类似 CDQ 分治处理点对关系的方式来处理它。假设现在正在处理的区间是 (l, r)，算法流程大致如下：

1. 如果 $l = r$，说明 dp_r 值的 max 部分已经被计算好了，直接令 $dp_r \leftarrow dp_r + 1$ 然后返回即可；
2. 递归使用 `solve(l,mid)`；
3. 处理所有 $l \le j \le mid$，$mid + 1 \le i \le r$ 的转移关系；
4. 递归使用 `solve(mid+1,r)`。

第三步的做法与 CDQ 分治求三维偏序差不多。处理 $l \le j \le mid$，$mid + 1 \le i \le r$ 的转移关系的时候，我们会发现已经不用管 $j < i$ 这个限制条件了。因此，我们依然先将所有的点 i 和点 j 按 a 值进行排序处理，然后用双指针的方式将 j 点插入到树状数组里，最后查一下前缀最大值更新一下 dp_i 就可以了。

转移过程的正确性 该 CDQ 写法和处理点对间关系的 CDQ 写法最大的不同就是处理 $l \le j \le mid$，$mid + 1 \le i \le r$ 的点对这一部分。处理点对间关系的 CDQ 写法中，这一部分放到哪里都是可以的。但是，在用 CDQ 分治优化 DP 的时候，这个流程却必须夹在 `solve(l,mid)`、`solve(mid+1,r)` 的中间。原因是 DP 的转移是**有序的**，它必须满足两个条件，否则就是不对的：

1. 用来计算 dp_i 的所有 dp_j 值都必须是已经计算完毕的，不能存在「半成品」；
2. 用来计算 dp_i 的所有 dp_j 值都必须能更新到 dp_i，不能存在没有更新到的 dp_j 值。

以 8 这个点为例，它的 DP 值是在 `solve(1,8)`、`solve(5,8)`、`solve(7,8)` 这 3 个函数中更新完成的，而三次用来更新它的点分别是 $(1,4)$、$(5,6)$、$(7,7)$ 这三个不相交的区间。仔细观察就会发现，一个 i 点的 DP 值被更新了 log 次，而且，更新它的区间刚好是 $(1, i)$ 在线段树上被拆分出来的 log 个区间。因此，我们的确保证了所有合法的 j 都更新过点 i，满足第 2 个条件。

接着分析算法的执行流程：每次 `solve(l,r)` 结束的时候，(l,r) 区间的 DP 值会被全部计算好。由于我们每一次执行转移函数的时候，`solve(l,mid)` 已经结束，因此我们每一次执行的转移过程都是合法的，满足第 1 个条件。

如果把 CDQ 分治的递归树看成一颗线段树，那么 CDQ 分治就是这个线段树的**中序遍历函数**，因此我们相当于按顺序处理了所有的 DP 值，只是转移顺序被拆开了而已，所以算法是正确的。

将动态问题转化为静态问题

前两种情况使用 CDQ 分治的目的是将序列折半之后递归处理点对间的关系，来获得良好的复杂度。不过在本节中，折半的不是一般的序列，而是**时间序列**。

它适用于一些「需要支持做 xxx 修改然后做 xxx 询问」的数据结构题。该类题目有两个特点：

- 如果把询问离线，所有操作会按照时间自然地排成一个序列。
- 每一个修改均与之后的询问操作息息相关。而这样的「修改-询问」关系一共会有 $O(n^2)$ 对。

我们可以使用 CDQ 分治对于这个操作序列进行分治，处理修改和询问之间的关系。与处理点对关系的 CDQ 分治类似，假设正在分治的序列是 (l,r)，我们先递归地处理 (l,mid) 和 $(mid+1,r)$ 之间的修改-询问关系，再处理所有 $l \le i \le mid$，$mid+1 \le j \le r$ 的修改-询问关系，其中 i 是一个修改，j 是一个询问。

注意，如果各个修改之间是**独立**的话，我们无需处理 `solve(l,mid)` 和 `solve(mid+1,r)` 之间的时序关系（比如普通的加减法问题）。但是如果各个修改之间并不独立（比如说赋值操作），做完这个修改后，序列长什么样可能依赖于之前的序列。此时处理所有跨越 mid 的修改-询问关系的步骤就必须放在 `solve(l,mid)` 和 `solve(mid+1,r)` 之间。理由和 CDQ 分治优化 1D/1D 动态规划的原因是一样的：按照中序遍历序进行分治才能保证每一个修改都是严格按照时间顺序执行的。

例题：动态逆序对 给出 $1 \sim n$ 的一个排列，按照某种顺序依次删除 m 个元素，你的任务是在每次删除一个元素之前统计整个序列的逆序对数。

思路是把删除倒过来看成插入：设第 i 个被删除的元素的出现时间为 $m-i+1$，于是每个元素都有（位置，值，时间）三个属性。一个元素在它被插入时产生的逆序对，就是「位置在它前面且值比它大」以及「位置在它后面且值比它小」的、且时间比它早的元素个数，两者都是三维偏序，用 CDQ 分治统计即可。把答案倒着累加就能得到每次删除前的逆序对数。

习题

- P3810 【模板】三维偏序（陌上花开）

- P3157 [CQOI2011] 动态逆序对
- P2487 [SDOI2011] 拦截导弹
- P4390 [BOI2007] Mokia 摩基亚

参考资料与注释

1. OI Wiki：CDQ 分治
2. 陈丹琦，从《*Cash*》谈一类分治算法的应用

9.4 int128

```cpp
#include <bits/stdc++.h>
using namespace std;
// 提醒：在 C++ 中，__int128_t 和 __int128 是等价的，一般写 __int128 即可。
inline __int128 read() {
    __int128 x = 0;
    bool f = 0;
    char ch = getchar();
    while (ch < '0' || ch > '9') { if (ch == '-') f = 1; ch = getchar(); }
    while (ch >= '0' && ch <= '9') { x = (x << 1) + (x << 3) + (ch ^ 48);
        ch = getchar(); }
    return f ? -x : x;
}
inline void print(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}
inline void println(__int128 x) {
    print(x);
    putchar('\n');
}
// 场景：求 (a * b) % mod，其中 a, b, mod 都是 10^18 级别。
// 直接强转 __int128，算完再转回来，绝对不溢出！
long long mul_mod(long long a, long long b, long long mod) {
    // 强制转换为 __int128 后相乘，算完取模，最后再安全地转回 long long
    return (long long)((__int128)a * b % mod);
}
void int128_basics() {
    __int128 a = read();
```

```
    __int128 b = read();
    __int128 add = a + b;
    __int128 sub = a - b;
    __int128 mul = a * b;
    __int128 div = a / b;
    __int128 mod = a % b;
    __int128 bit_and = a & b;
    __int128 shift = a << 10;
    print(add); putchar(' '); println(mul);
}
inline __int128 abs_128(__int128 x) {
    return x < 0 ? -x : x;
}
__int128 gcd_128(__int128 a, __int128 b) {
    return b == 0 ? a : gcd_128(b, a % b);
}

void check_limits() {
    // __int128 的最大值大概是 1.7 * 10^38
    __int128 max_val = ((__int128)1 << 126) - 1 + ((__int128)1 << 126);
    cout << "Max __int128 is roughly: ";
    println(max_val);
    // 输出: 170141183460469231731687303715884105727
}
```

9.5 modint

```
#include <bits/stdc++.h>
using namespace std;

// ============================================
// 1. 取模类
// ============================================
// MOD 作为模板参数，便于编译期常数优化。
template <int MOD>
struct ModInt {
    int val;

    // 构造函数：把值归一到 [0, MOD) 并防溢出
    ModInt(long long v = 0) {
        if (v < 0) v = v % MOD + MOD;
        if (v >= MOD) v %= MOD;
        val = v;
```

```
17      }
18
19      // 四则运算，结果均已取模
20      ModInt& operator+=(const ModInt& rhs) {
21          val += rhs.val;
22          if (val >= MOD) val -= MOD; // 用减法代替取模
23          return *this;
24      }
25      ModInt& operator-=(const ModInt& rhs) {
26          val -= rhs.val;
27          if (val < 0) val += MOD;
28          return *this;
29      }
30      ModInt& operator*=(const ModInt& rhs) {
31          val = 1ll * val * rhs.val % MOD;
32          return *this;
33      }
34      ModInt& operator/=(const ModInt& rhs) {
35          // 除法转为乘逆元，要求 MOD 为素数
36          return *this *= rhs.inv();
37      }
38
39      // 四则运算符
40      friend ModInt operator+(ModInt lhs, const ModInt& rhs) { return lhs +=
            rhs; }
41      friend ModInt operator-(ModInt lhs, const ModInt& rhs) { return lhs -=
            rhs; }
42      friend ModInt operator*(ModInt lhs, const ModInt& rhs) { return lhs *=
            rhs; }
43      friend ModInt operator/(ModInt lhs, const ModInt& rhs) { return lhs /=
            rhs; }
44
45      // 一元运算符：取负、自增、自减
46      ModInt operator-() const { return ModInt(-val); }
47      ModInt& operator++() { return *this += 1; }
48      ModInt operator++(int) { ModInt tmp = *this; ++*this; return tmp; }
49      ModInt& operator--() { return *this -= 1; }
50      ModInt operator--(int) { ModInt tmp = *this; --*this; return tmp; }
51
52      // 比较运算符
53      bool operator==(const ModInt& rhs) const { return val == rhs.val; }
54      bool operator!=(const ModInt& rhs) const { return val != rhs.val; }
55
```

```
    // 快速幂，O(log k)
    ModInt pow(long long k) const {
        ModInt res = 1, a = *this;
        while (k > 0) {
            if (k & 1) res *= a;
            a *= a;
            k >>= 1;
        }
        return res;
    }

    // 求逆元：费马小定理，要求 MOD 为素数
    ModInt inv() const {
        // a^(p-2) 即 a 的逆元 (mod p)
        return pow(MOD - 2);
    }

    // 输入输出重载
    friend istream& operator>>(istream& is, ModInt& m) {
        long long v; is >> v; m = ModInt(v); return is;
    }
    friend ostream& operator<<(ostream& os, const ModInt& m) {
        return os << m.val;
    }
};

// ==========================================
// 2. 使用示例
// ==========================================
// 998244353 与 1e9+7 是常用模数
using mint = ModInt<998244353>;
// using mint = ModInt<1000000007>;

void modint_example() {
    // 1. 读入
    mint a, b;
    // 假设输入 998244354 2
    // cin >> a >> b;
    a = 998244354; // 自动归约为 1
    b = 2;

    // 2. 各运算符结果均已取模
    mint sum = a + b;       // 1 + 2 = 3
```

```
    mint sub = a - b;        // 1 - 2 = -1, 自动转为 998244352
    mint mul = a * b * 10;   // 连乘不溢出

    // 3. 除法自动求逆元
    mint div = a / b;        // 等价于 1 * inv(2)

    // 4. 快速幂
    mint ans = a.pow(10086);

    // cout << sum << " " << div << endl;
}
```